

DATABASE INTERVIEW REFERENCE

Microsoft SQL Server Interview Prep Guide

55 Essential Topics for SQL Server Professionals

A consolidated, exam-style reference covering T-SQL fundamentals, schema design, indexing and query optimization, transactions & concurrency control, HA/DR with Always On, partitioning, security, monitoring, and the common interview, scenario, and HR questions you are likely to face — each with definitions, code examples, and trade-offs you can recite in 60 seconds.

T-SQL

Joins

CTE

Indexes

Stored Procs

Triggers

Transactions

ACID

Locks

Isolation

Deadlocks

Execution Plans

Always On

Partitioning

Security

TOPICS
COVERED**55 Sections**

FORMAT

**Concept + Code +
Q&A**

TARGET VERSION

**SQL Server 2019+ /
2022**

Table of Contents

Foreword	10
1. Introduction to MSSQL	11
1.1 Key Capabilities	11
1.2 SQL Server Releases (Recent)	11
1.3 Tools You Should Know	11
2. SQL Server Architecture	12
2.1 High-Level Components	12
2.2 Database File Structure	12
2.3 Page and Extent	13
2.4 Buffer Pool and Checkpoint	13
2.5 Transaction Log & Recovery	13
3. SQL Server Editions	13
3.1 Edition Comparison	13
3.2 Enterprise-Only Features (Highlights)	14
4. Databases and Tables	14
4.1 System Databases	14
4.2 Creating a Database	15
4.3 Creating Tables	15
4.4 Schema-Bound Object Naming	15
4.5 ALTER TABLE Operations	15
5. Data Types	16
5.1 Numeric Types	16
5.2 Character Types	17
5.3 Date & Time Types	17
5.4 Binary & Special Types	17
6. Constraints	18
6.1 Constraint Types	18
6.2 Primary Key Examples	18
6.3 Foreign Key Actions	19
6.4 CHECK Constraints	19
7. CRUD Operations	19

7.1 INSERT	19
7.2 SELECT	20
7.3 UPDATE	20
7.4 DELETE	20
7.5 TRUNCATE vs DELETE	21
7.6 MERGE (Upsert)	21
8. SQL Clauses	21
8.1 Logical Execution Order	21
8.2 WHERE Operators	22
8.3 LIKE Patterns	22
8.4 ORDER BY	23
8.5 OFFSET / FETCH (Modern Pagination)	23
9. Joins	23
9.1 Join Types	23
9.2 INNER JOIN Example	24
9.3 LEFT JOIN with NULL Check	24
9.4 Self-Join (Hierarchy)	24
9.5 CROSS APPLY (Per-Row TVF)	24
10. Set Operators	24
10.1 The Four Operators	24
10.2 Examples	25
10.3 Rules	25
11. Functions (Built-in)	25
11.1 Function Categories	25
11.2 NULL-Handling Functions (Critical)	26
11.3 Conversion Functions	26
11.4 System Functions	26
12. String Functions	27
12.1 Common String Functions	27
12.2 Examples	28
13. Date Functions	28
13.1 Getting the Current Date/Time	28
13.2 Date Arithmetic	28
13.3 Extracting Components	29

13.4 Date Construction & Truncation	29
14. Aggregate Functions	29
14.1 Built-in Aggregates	30
14.2 NULL Behaviour (Critical)	30
14.3 Examples	30
14.4 DISTINCT Inside Aggregates	31
15. GROUP BY and HAVING	31
15.1 Basic GROUP BY	31
15.2 HAVING (Filter on Aggregates)	31
15.3 GROUP BY with Multiple Columns	32
15.4 GROUPING SETS / ROLLUP / CUBE	32
15.5 GROUPING() Function	32
16. Subqueries	33
16.1 Subquery Categories	33
16.2 Scalar Subquery	33
16.3 IN / NOT IN (Multi-row)	33
16.4 Correlated Subquery	33
16.5 EXISTS / NOT EXISTS	34
16.6 Derived Table (Subquery in FROM)	34
17. Common Table Expressions (CTE)	34
17.1 Basic (Non-recursive) CTE	34
17.2 Multiple CTEs	35
17.3 Recursive CTE — Hierarchy Walk	35
17.4 Recursive CTE — Date Series	35
18. Temporary Tables	36
18.1 Local vs Global Temp Tables	36
18.2 Creating and Using	36
18.3 Indexes on Temp Tables	36
18.4 When to Use Temp Tables	37
19. Table Variables	37
19.1 Declaration and Use	37
19.2 Table Variable vs Temp Table	37
19.3 Table-Valued Parameters (TVPs)	38
20. Views	38

20.1 Creating a View	38
20.2 SCHEMABINDING	39
20.3 Indexed Views (Materialised Views)	39
20.4 View Limitations	39
21. Indexes	39
21.1 Clustered vs Nonclustered	40
21.2 Creating Indexes	40
21.3 INCLUDE vs Key Columns	40
21.4 How Indexes Are Used	41
21.5 Index Maintenance	41
22. Stored Procedures	41
22.1 Anatomy	41
22.2 Why Use Stored Procedures	42
22.3 Parameter Sniffing	42
22.4 OUTPUT Parameters	43
23. User Defined Functions (UDF)	43
23.1 UDF Types	43
23.2 Scalar UDF (and Inlining)	43
23.3 Inline TVF (Preferred for Table-Returning)	44
23.4 Multi-Statement TVF (Avoid if Possible)	44
24. Triggers	44
24.1 DML Trigger Types	45
24.2 inserted and deleted Pseudo-Tables	45
24.3 AFTER Trigger Example (Audit Log)	45
24.4 INSTEAD OF Trigger (Update through View)	45
24.5 Trigger Pitfalls	46
25. Transactions	46
25.1 Explicit Transactions	46
25.2 Transaction Modes	47
25.3 SAVE TRAN (Partial Rollback)	47
25.4 XACT_ABORT and Error Handling	47
26. ACID Properties	48
26.1 The Four Properties	48
26.2 Atomicity in Detail	48

26.3 Consistency in Detail	48
26.4 Isolation in Detail	48
26.5 Durability in Detail	49
27. Locks and Blocking	49
27.1 Lock Modes	49
27.2 Lock Granularity	49
27.3 Blocking vs Deadlocking	50
27.4 Investigating Blocking	50
27.5 Reducing Blocking	50
28. Isolation Levels	51
28.1 The Five Levels	51
28.2 Setting Isolation Level	51
28.3 Read Anomalies (What Each Level Prevents)	51
28.4 SNAPSHOT and RCSI (Optimistic Concurrency)	52
29. Deadlocks	52
29.1 Classic Deadlock Scenario	52
29.2 How SQL Server Detects Deadlocks	52
29.3 Capturing Deadlocks	53
29.4 Preventing Deadlocks	53
29.5 Application Retry Pattern	53
30. Execution Plans	54
30.1 Estimated vs Actual Plans	54
30.2 Generating Plans	54
30.3 Common Operators	54
30.4 Reading a Plan	55
30.5 Missing Index Hints	55
31. Query Optimization	56
31.1 Sargability (Search Argument-able)	56
31.2 Common Optimisation Techniques	56
31.3 Temp Table vs CTE for Performance	57
31.4 Window Functions vs Self-Joins	57
32. Normalization	57
32.1 Normal Forms	57
32.2 1NF Example	57

32.3 2NF Example	58
32.4 3NF Example	58
33. Denormalization	59
33.1 When to Denormalise	59
33.2 Techniques	59
33.3 Denormalisation Trade-offs	59
33.4 Example: Snapshot Column	59
34. Keys in SQL Server	60
34.1 Key Types	60
34.2 Surrogate vs Natural Keys	60
34.3 GUIDs as Primary Keys	61
34.4 Composite Keys	61
35. Identity Column	61
35.1 Basic Syntax	61
35.2 Retrieving Generated Values	62
35.3 IDENTITY Insert and Reseed	62
35.4 IDENTITY Limitations	62
36. Sequences	62
36.1 Creating and Using	62
36.2 Sequence vs IDENTITY	63
36.3 Use Cases	63
37. Pagination	64
37.1 OFFSET / FETCH (Recommended)	64
37.2 ROW_NUMBER (Older Pattern)	64
37.3 Keyset Pagination (High-Performance)	64
37.4 Returning Total Count	64
38. Dynamic SQL	65
38.1 Basic Construction	65
38.2 sp_executesql vs EXEC	65
38.3 Dynamic Search Proc	65
38.4 SQL Injection Prevention	66
39. XML and JSON	66
39.1 XML Data Type	66
39.2 JSON in SQL Server	66

39.3 XML vs JSON	67
40. Error Handling	67
40.1 TRY...CATCH Structure	68
40.2 ERROR_* Functions	68
40.3 THROW vs RAISERROR	68
40.4 Custom Error Messages	69
40.5 XACT_ABORT and Transaction State	69
41. Backup and Restore	69
41.1 Backup Types	69
41.2 Recovery Models	70
41.3 Backup Commands	70
41.4 Restore Sequence	70
41.5 Common Backup Strategy (OLTP)	71
42. SQL Server Agent	71
42.1 Core Concepts	71
42.2 Creating a Job in T-SQL	71
42.3 Job History and Troubleshooting	72
42.4 Common Job Categories	72
43. Security	73
43.1 Security Layers	73
43.2 Authentication Modes	73
43.3 Encryption Technologies	73
43.4 Always Encrypted Example	74
43.5 Row-Level Security	74
44. Permissions and Roles	75
44.1 Fixed Server Roles	75
44.2 Fixed Database Roles	75
44.3 GRANT / DENY / REVOKE	76
44.4 Application Roles	76
44.5 Principle of Least Privilege	76
45. Replication	76
45.1 Replication Types	77
45.2 Replication Architecture	77
45.3 When to Use Replication (vs Alternatives)	77

45.4 Common Issues	77
46. Always On Availability Groups	78
46.1 AG Architecture	78
46.2 AG vs Failover Cluster Instance (FCI)	78
46.3 Creating an AG (T-SQL)	78
47. Partitioning	79
47.1 Why Partition?	79
47.2 Three Objects	80
47.3 Creating a Partitioned Table	80
47.4 Switching Partitions (Fast Archival)	80
47.5 Partition Elimination	81
48. SQL Profiler and Monitoring	81
48.1 Tools Comparison	81
48.2 Key DMVs	81
48.3 Wait Statistics	82
48.4 Active Sessions and Blocks	82
48.5 Query Store	82
51. Real-Time Optimization Examples	83
51.1 Non-Sargable Date Filter	83
51.2 Key Lookup Elimination	83
51.3 Correlated Subquery → Window Function	83
51.4 Cursor → Set-Based UPDATE	83
51.5 Parameter Sniffing Fix	84
51.6 Temp Table vs CTE for Multi-Reference	84
52. Best Practices	84
52.1 Schema Design	85
52.2 Query Writing	85
52.3 Indexing	85
52.4 Transactions	85
52.5 Security	86
52.6 Operations	86
53. Frequently Asked Queries	86
53.1 Find Duplicate Rows	86
53.2 Delete Duplicates (Keep One)	87

53.3 Nth Highest Salary	87
53.4 Top N per Group	87
53.5 Running Total	87
53.6 Year-over-Year Growth	87
53.7 Pivoting Rows to Columns	87
53.8 Hierarchical Query (Recursive CTE)	88
53.9 Find Missing IDs (Gaps)	88
53.10 Concatenate Rows into a String	88
54. Advanced Topics	88
54.1 Columnstore Indexes	88
54.2 In-Memory OLTP (Hekaton)	89
54.3 Query Hints	89
54.4 Service Broker	89
54.5 Change Data Capture (CDC) and Change Tracking	90
54.6 Temporal Tables (System-Versioned)	90
54.7 Graph Tables (SQL Server 2017+)	90
55. HR + Project Related Questions	90
55.1 STAR Framework	90
55.4 Questions to Ask the Interviewer	91
55.5 Final Tips	91

Foreword

This guide consolidates 55 essential topics that a Microsoft SQL Server professional is expected to know in interviews and day-to-day database work. Each topic is structured as a concise definition, the core concepts you must internalise, T-SQL syntax examples, common pitfalls, and one or two interview-style questions with model answers. The aim is to give you a single revision document that you can read end-to-end in a few hours and revisit before any interview round.

The material reflects the realistic expectations of mid-level SQL Server roles: solid T-SQL fluency (joins, CTEs, window functions), schema design (normalisation, keys, constraints), performance tuning (indexes, execution plans, query optimisation), concurrency control (locks, isolation levels, deadlocks), and production operations (backups, Always On, replication, security). The closing sections focus on common interview questions, scenario-based problems, real-time optimisation case studies, and HR/project questions you are likely to face.

How to use this guide: Read each topic in order; do not skip the T-SQL examples even if the concept feels familiar. Most interview rejections happen because a candidate can describe a concept but cannot write the syntax. After finishing a topic, attempt to write the example from memory and compare it with the printed version. Pay extra attention to execution plans, isolation levels, and index design — these are the highest-frequency mid-level interview topics.

1. Introduction to MSSQL

Microsoft SQL Server (MSSQL) is a relational database management system (RDBMS) developed by Microsoft. It uses T-SQL (Transact-SQL) — an extension of ANSI SQL — for querying and managing data. SQL Server runs primarily on Windows, but since SQL Server 2017 it also runs on Linux and in Docker containers, and the same engine powers Azure SQL Database (PaaS), Azure SQL Managed Instance, and Azure Synapse Analytics.

1.1 Key Capabilities

- **Relational storage** — Tables, schemas, constraints, foreign keys, normalised design.
- **Programmability** — Stored procedures, user-defined functions, triggers, CLR integration.
- **Transaction processing** — Full ACID compliance with multiple isolation levels.
- **High availability** — Always On Availability Groups, failover cluster instances, log shipping.
- **Disaster recovery** — Full / differential / log backups, point-in-time restore.
- **Security** — Authentication (Windows / SQL logins), authorisation (roles), encryption (TDE, Always Encrypted), row-level security, dynamic data masking.
- **Business intelligence** — Integration Services (ETL), Analysis Services (OLAP), Reporting Services (paginated reports).
- **Performance** — Query optimiser, execution plans, columnstore indexes, in-memory OLTP.

1.2 SQL Server Releases (Recent)

Version	Headline Features	Status
2014	In-memory OLTP (Hekaton), clustered columnstore updateable	Older
2016	Always Encrypted, R integration, Query Store, Stretch DB	Still seen in legacy
2017	Linux support, Docker, Python integration, Adaptive Query Processing	Common
2019	Big Data Clusters, Intelligent Query Processing, Accelerated DB Recovery	Common
2022	Azure Synapse link, Parameter Sensitive Plan Optimization, Ledger	Latest LTS
2025	Managed disk pools, improved columnstore, AI features	Newest

1.3 Tools You Should Know

- **SQL Server Management Studio (SSMS)** — The classic GUI for querying and managing SQL Server. Windows-only, free.
- **Azure Data Studio** — Cross-platform (Electron-based) notebook-friendly editor; good for Linux/macOS.

- **sqlcmd** — Command-line query tool — useful for scripting and CI.
- **SQL Server Profiler / Extended Events** — Trace server activity; Profiler is deprecated, use Extended Events.
- **Database Engine Tuning Advisor (DTA)** — Recommends indexes and partitions; less used now — Query Store covers most cases.
- **bcp** — Bulk copy program for fast import/export of data files.

2. SQL Server Architecture

SQL Server is a client/server system. A single instance hosts multiple databases, each with its own files, logins, and users. The engine is composed of a Network Interface layer, a Query Processor (relational engine), a Storage Engine, and the SQLOS (scheduling and memory manager). Understanding this architecture helps you reason about performance, locking, and recovery.

2.1 High-Level Components

Component	Function	Notes
Network Interface	Tabular Data Stream (TDS) protocol over TCP/IP / named pipes	Client↔server protocol
Relational Engine	Parses, compiles, optimises, and executes queries	Query Processor
Storage Engine	Manages data files, filegroups, pages, extents, buffers	Disk↔memory bridge
SQLOS	Scheduling, memory, I/O, exception handling	User-mode OS layer
Buffer Pool	In-memory cache of recently-used data pages	Why reads are fast
Plan Cache	Cached query plans and execution trees	Why recompiles matter
Log Manager	Write-ahead logging to transaction log	ACID & recovery

2.2 Database File Structure

- **Primary data file (.mdf)** — Required; contains database boot info and may hold user data. Each database has exactly one .mdf.
- **Secondary data files (.ndf)** — Optional; spread data across files/filegroups for parallelism. Useful on multi-disk systems.
- **Transaction log file (.ldf)** — Required; sequential log of every transaction. Used for crash recovery, replication, point-in-time restore.
- **Filegroups** — Logical grouping of files. Primary filegroup holds system tables. User filegroups let you place large tables on faster disks or partition by filegroup.

2.3 Page and Extent

Unit	Size	Purpose	Types
Page	8 KB	Smallest unit of I/O	Data, index, GAM, SGAM, PFS, IAM, log
Extent	8 pages = 64 KB	Unit of space allocation	Uniform (1 object) or Mixed (up to 8)
IAM page	1 per allocation unit	Maps which extents an object uses	First check on scan
PFS page	1 per ~64 MB	Free space per page (0-100%)	Used for INSERT/UPDATE

2.4 Buffer Pool and Checkpoint

When SQL Server reads a page from disk, it pulls it into the buffer pool (RAM). Subsequent reads of the same page hit memory. Modifications write to the buffer (dirty page) and synchronously to the transaction log (write-ahead logging). The dirty page is later written to the .mdf by a checkpoint or lazy writer. This is why transaction log writes (sequential) dominate commit latency, not data file writes (random).

2.5 Transaction Log & Recovery

- **Write-ahead logging (WAL)** — Log record written and flushed to .ldf BEFORE the data page is written to .mdf. Guarantees durability.
- **Checkpoint** — Writes all dirty pages from buffer pool to .mdf and truncates the active portion of the log (in SIMPLE recovery).
- **Recovery models** — SIMPLE: log truncated on checkpoint (no point-in-time restore). FULL: log preserved until log backup (supports PITR). BULK_LOGGED: minimal logging for bulk operations.
- **VLF (Virtual Log File)** — The .ldf is split into VLFs. Too many small VLFs (10,000+) slows startup; fix with proper autogrowth sizing.

3. SQL Server Editions

Microsoft ships SQL Server in several editions with different feature sets, licensing, and resource limits. Knowing the edition tells you what features you can use (Always On, TDE, partitioning) and how much RAM / CPU / DB size the instance supports.

3.1 Edition Comparison

Edition	Cost	Limitations	Best For
Express	Free	10 GB DB size, 1 GB RAM, 1 socket/4 cores	Dev, small apps, learning
Developer	Free	Full feature set, but licensed for dev/test only	Dev & test environments

Edition	Cost	Limitations	Best For
Evaluation	Free, 180 days	Enterprise features	Proof of concepts
Web	Paid (hosters)	Limited for shared hosting	ISP-hosted web apps
Standard	Paid	Most features, lower HA/scalability limits	Mid-sized business apps
Enterprise	Paid	All features, no resource caps	Mission-critical workloads

3.2 Enterprise-Only Features (Highlights)

- **Always On Availability Groups** — Up to 8 secondaries (Standard = 1), readable secondaries, multiple databases per AG.
- **Transparent Data Encryption (TDE)** — Encrypts data files at rest.
- **Columnstore indexes** — Standard has read-only columnstore on disk; Enterprise has updateable + in-memory columnstore.
- **Table & index partitioning** — Split large tables across filegroups; Enterprise only.
- **In-Memory OLTP** — Memory-optimised tables and natively compiled stored procedures.
- **Resource Governor** — Cap CPU/memory per workload group.
- **Parallelism on index rebuilds** — Enterprise uses multiple cores; Standard is single-threaded.
- **Online index operations** — Rebuild indexes while users modify the table.

4. Databases and Tables

A SQL Server instance hosts system databases and one or more user databases. Each database contains schemas, tables, views, stored procedures, and other objects. Knowing how to create, alter, and manage databases and tables is the foundation of every SQL Server role.

4.1 System Databases

Database	Purpose	Notes
master	Instance metadata: logins, server config, database file locations	Back up after every change
model	Template for new databases	Customise default sizes
msdb	SQL Agent jobs, backups history, maintenance plans, alerts	Critical for restore history
tempdb	Temporary tables, table variables, sorts, row versions	Re-created on every restart
resource (hidden)	System stored procedures and views (sys.*)	Read-only
distribution	Only on replication Distributor	Replication metadata

4.2 Creating a Database

```
CREATE DATABASE SalesDB
ON PRIMARY
    ( NAME = N'SalesDB_data',
      FILENAME = N'D:\Data\SalesDB.mdf',
      SIZE = 100 MB, MAXSIZE = 10 GB, FILEGROWTH = 50 MB ),
    ( NAME = N'SalesDB_data2',
      FILENAME = N'E:\Data\SalesDB2.ndf',
      SIZE = 100 MB, FILEGROWTH = 50 MB )
LOG ON
    ( NAME = N'SalesDB_log',
      FILENAME = N'L\Log\SalesDB_log.ldf',
      SIZE = 50 MB, MAXSIZE = 5 GB, FILEGROWTH = 25 MB );
GO
```

Autogrowth best practice: Use fixed growth sizes (e.g., 50-100 MB for data, 25 MB for log) rather than percentages. Default 1 MB autogrowth causes thousands of small growths on a busy database — each is a brief block. Pre-size for expected capacity and enable Instant File Initialization (security policy Perform Volume Maintenance Tasks) to skip zeroing data files on growth.

4.3 Creating Tables

```
CREATE SCHEMA Sales;
GO

CREATE TABLE Sales.Customers (
    CustomerID INT IDENTITY(1,1) NOT NULL,
    CustomerCode VARCHAR(20) NOT NULL,
    Name NVARCHAR(100) NOT NULL,
    Email NVARCHAR(255) NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    UpdatedAt DATETIME2 NULL,
    CONSTRAINT PK_Customers PRIMARY KEY CLUSTERED (CustomerID),
    CONSTRAINT UQ_Customers_Code UNIQUE (CustomerCode)
);
GO
```

4.4 Schema-Bound Object Naming

- **Two-part names** — Always use Schema.ObjectName (e.g., Sales.Customers). One-part names rely on default schema and break on different logins.
- **Schemas** — Logical namespace for objects. Default is dbo. Use schemas to group related tables (Sales, HR, Inventory) and grant permissions at schema level.
- **Object name limits** — 128 chars for regular identifiers; 116 for temp tables; cannot start with @, #, ##.

4.5 ALTER TABLE Operations

```
-- Add a column
ALTER TABLE Sales.Customers ADD Region NVARCHAR(50) NULL;

-- Add a default constraint
ALTER TABLE Sales.Customers
    ADD CONSTRAINT DF_Customers_Region DEFAULT N'Unknown' FOR Region;

-- Alter a column type/size (may rewrite the table)
```



```

ALTER TABLE Sales.Customers ALTER COLUMN Name NVARCHAR(200) NOT NULL;

-- Drop a column
ALTER TABLE Sales.Customers DROP COLUMN Email;

-- Drop a constraint
ALTER TABLE Sales.Customers DROP CONSTRAINT UQ_Customers_Code;

```

ALTER COLUMN gotcha: Changing a column type or nullability can rewrite the entire table and rebuild all indexes — locks the table for hours on a 100M-row table. Use the new (SQL Server 2022+) `ALTER TABLE ... ADD COLUMN ... WITH (DEFAULT)` pattern that adds a constant default as metadata only (instant).

5. Data Types

Choosing the right data type affects storage, performance, accuracy, and correctness. SQL Server offers ~40 data types across numeric, character, date, binary, and special categories. Pick the smallest type that fits; never use `NVARCHAR(MAX)` for a 50-char field.

5.1 Numeric Types

Type	Storage	Range / Notes	Use For
BIT	1 bit (rounded to 1 byte)	0/1/NULL	Boolean flags
TINYINT	1 byte	0-255	Small enums, status codes
SMALLINT	2 bytes	-32,768 to 32,767	Small counters
INT	4 bytes	-2.1B to 2.1B	Most IDs, counters
BIGINT	8 bytes	-9.2E18 to 9.2E18	Large IDs, timestamps
DECIMAL(p,s)	5-17 bytes	Exact numeric	Money, scientific
NUMERIC(p,s)	same as DECIMAL	ANSI synonym	Same as DECIMAL
MONEY	8 bytes	-922 trillion to 922 trillion	Currency (4 decimals)
SMALLMONEY	4 bytes	-214,748 to 214,748	Small currency
REAL (FLOAT(24))	4 bytes	Approximate, 7 digits	Scientific
FLOAT(n)	8 bytes (n=53)	Approximate, 15 digits	Scientific

5.2 Character Types

Type	Storage	Notes	Use For
CHAR(n)	n bytes (padded)	Fixed-width ASCII	Country codes, fixed codes
VARCHAR(n)	actual length + 2	Variable ASCII, max 8000	English text
VARCHAR(MAX)	up to 2 GB	Large ASCII text	Long text, JSON
NCHAR(n)	2n bytes (padded)	Fixed Unicode (UCS-2)	Fixed-width i18n codes
NVARCHAR(n)	2*length + 2	Variable Unicode, max 4000 chars	Most text fields
NVARCHAR(MAX)	up to 2 GB	Large Unicode	Long multilingual text
TEXT / NTEXT	deprecated	Use VARCHAR(MAX) instead	Avoid — LOB pointer

5.3 Date & Time Types

Type	Storage	Precision / Range	Notes
DATE	3 bytes	Date only (no time)	1900-01-01 to 9999-12-31
TIME(n)	3-5 bytes	Time only, n = fractional seconds (0-7)	Scheduling
SMALLDATETIME	4 bytes	1-minute precision, 1900-2079	Legacy
DATETIME	8 bytes	3.33ms precision, 1753-9999	Legacy but common
DATETIME2(n)	6-8 bytes	100ns precision, larger range	Recommended
DATETIMEOFFSET	8-10 bytes	DATETIME2 + timezone offset	Global apps

Best practice: Use DATE for date-only, DATETIME2(0-7) for date+time. Avoid DATETIME (lower precision, larger storage, no timezone) for new designs. Use DATETIMEOFFSET when storing instants across timezones — store UTC and convert at presentation.

5.4 Binary & Special Types

- **BINARY(n) / VARBINARY(n)** — Fixed / variable binary up to 8000 bytes. Use for hashes, GUIDs (better as UNIQUEIDENTIFIER), encrypted values.
- **VARBINARY(MAX)** — Up to 2 GB. Use FILESTREAM to store on NTFS for files > 1 MB.
- **UNIQUEIDENTIFIER** — 16-byte GUID. Use NEWSEQUENTIALID() for clustered index keys (random GUIDs cause page splits).

- **XML** — Native XML type with XQuery support. Use sparingly — relational is usually better; store XML in NVARCHAR(MAX) for opaque blobs.
- **JSON** — No native type; store as NVARCHAR(MAX) and use OPENJSON / JSON_VALUE.
- **HIERARCHYID** — CLR type for tree structures (org charts, folder trees).
- **SQL_VARIANT** — Holds any type — anti-pattern, avoid in production.

6. Constraints

Constraints enforce data integrity at the database level. They are checked on every INSERT, UPDATE, DELETE and any violation rolls back the statement. Constraints are preferable to triggers or application checks because they cannot be bypassed.

6.1 Constraint Types

Constraint	Purpose	Typical Use
NOT NULL	Column cannot be NULL	Required fields
PRIMARY KEY	Unique + NOT NULL; creates clustered index by default	Row identifier
UNIQUE	Unique values; allows one NULL (per duplicate-filtered index)	Alternate keys
FOREIGN KEY	References PK/UNIQUE of another table; enforces referential integrity	Relationships
CHECK	Boolean expression on column or row	Domain integrity (age >= 0)
DEFAULT	Value used when INSERT omits the column	SYSUTCDATETIME() for created_at

6.2 Primary Key Examples

```
-- Single column clustered PK
CREATE TABLE dbo.Employees (
    EmployeeID INT IDENTITY(1,1) NOT NULL,
    CONSTRAINT PK_Employees PRIMARY KEY CLUSTERED (EmployeeID)
);

-- Composite PK (e.g., junction table)
CREATE TABLE dbo.OrderItems (
    OrderID INT NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
    CONSTRAINT PK_OrderItems PRIMARY KEY CLUSTERED (OrderID, ProductID),
    CONSTRAINT FK_OI_Orders FOREIGN KEY (OrderID) REFERENCES dbo.Orders(OrderID),
    CONSTRAINT FK_OI_Products FOREIGN KEY (ProductID) REFERENCES
dbo.Products(ProductID)
);
```

6.3 Foreign Key Actions

- **ON DELETE NO ACTION (default)** — Restrict delete of parent if children exist.
- **ON DELETE CASCADE** — Delete children automatically when parent deleted. Use cautiously — can cascade through several levels and delete a lot of data.
- **ON DELETE SET NULL** — Set child FK column to NULL when parent deleted (column must allow NULL).
- **ON DELETE SET DEFAULT** — Set to default value (must have DEFAULT constraint).
- **ON UPDATE actions** — Same options — typically NO ACTION or CASCADE for natural keys that may be updated.

6.4 CHECK Constraints

```
-- Column-level
CREATE TABLE dbo.Accounts (
    AccountID INT IDENTITY PRIMARY KEY,
    Balance DECIMAL(18,2) NOT NULL,
    OpenedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    CONSTRAINT CK_Accounts_Balance CHECK (Balance >= 0),
    CONSTRAINT CK_Accounts_OpenedAt CHECK (OpenedAt <= SYSUTCDATETIME())
);

-- Add CHECK to existing table (use WITH NOCHECK to skip existing data)
ALTER TABLE dbo.Accounts WITH NOCHECK
    ADD CONSTRAINT CK_Accounts_Balance CHECK (Balance >= 0);
```

CHECK vs trigger: CHECK is simpler and faster (optimizer uses it for contradiction detection). Use triggers only when the rule spans multiple tables or needs procedural logic. CHECK cannot reference other rows or tables; triggers can. Mark a CHECK with NOCHECK to apply it going forward without validating existing data — but this disables some optimizer optimizations on that table.

7. CRUD Operations

CRUD = Create, Read, Update, Delete — the four basic data manipulation operations. They map to INSERT, SELECT, UPDATE, DELETE in SQL. Mastering the syntax and the common patterns (multi-row insert, OUTPUT clause, MERGE) is fundamental.

7.1 INSERT

```
-- Single row
INSERT INTO dbo.Customers (Name, Email, Country)
VALUES (N'Acme Corp', N'sales@acme.com', N'US');

-- Multi-row (one statement, one log entry – efficient)
INSERT INTO dbo.Customers (Name, Email, Country)
VALUES (N'A', N'a@x.com', N'US'),
       (N'B', N'b@x.com', N'UK'),
       (N'C', N'c@x.com', N'IN');

-- INSERT ... SELECT (from another table)
INSERT INTO dbo.Customers_Archive (CustomerID, Name, ArchivedAt)
```

```

SELECT CustomerID, Name, SYSUTCDATETIME()
FROM dbo.Customers
WHERE IsActive = 0;

-- INSERT ... EXEC (capture stored proc output)
INSERT INTO dbo.TempResults
EXEC dbo.GetDailySales @Date = '2026-07-01';

-- SELECT INTO (creates new table)
SELECT CustomerID, Name, Country
INTO dbo.Customers_Backup
FROM dbo.Customers
WHERE Country = N'US';

```

7.2 SELECT

```

-- Basic
SELECT CustomerID, Name, Email
FROM dbo.Customers
WHERE Country = N'US'
ORDER BY Name
OFFSET 0 ROWS FETCH NEXT 20 ROWS ONLY; -- modern pagination

-- DISTINCT
SELECT DISTINCT Country FROM dbo.Customers ORDER BY Country;

-- TOP with ties
SELECT TOP (10) WITH TIES ProductID, COUNT(*) AS OrderCount
FROM dbo.OrderItems
GROUP BY ProductID
ORDER BY OrderCount DESC;

```

7.3 UPDATE

```

-- Simple
UPDATE dbo.Customers
SET Email = N'new@acme.com', UpdatedAt = SYSUTCDATETIME()
WHERE CustomerID = 42;

-- UPDATE with FROM (join)
UPDATE c
SET c.Status = N'VIP'
FROM dbo.Customers c
JOIN dbo.Orders o ON o.CustomerID = c.CustomerID
WHERE o.OrderDate >= DATEADD(year, -1, SYSUTCDATETIME())
GROUP BY c.CustomerID, c.Status
HAVING SUM(o.Total) > 10000;

-- UPDATE with OUTPUT (capture before/after values)
UPDATE dbo.Products
SET Price = Price * 1.10
OUTPUT inserted.ProductID, deleted.Price AS OldPrice, inserted.Price AS NewPrice
WHERE CategoryID = 5;

```

7.4 DELETE

```

-- Simple
DELETE FROM dbo.Customers WHERE CustomerID = 42;

-- DELETE with JOIN

```

```

DELETE c
FROM dbo.Customers c
LEFT JOIN dbo.Orders o ON o.CustomerID = c.CustomerID
WHERE o.OrderID IS NULL
      AND c.CreatedAt < DATEADD(year, -5, SYSUTCDATETIME());

-- TRUNCATE: fast, minimal logging, resets IDENTITY, cannot be rolled back in a
-- transaction at the row level (operation is logged at page level)
TRUNCATE TABLE dbo.StagingLog;

```

7.5 TRUNCATE vs DELETE

Operation	Mechanism	Resets IDENTITY	Fires Triggers	Works with WHERE
TRUNCATE	Page-level deallocation; minimal log	Yes	Yes (faster)	Yes
DELETE	Row-by-row; full log per row	Yes	No	No

TRUNCATE caveats: Cannot truncate a table referenced by a FOREIGN KEY. Cannot truncate a table in a transaction that is being replicated. Fires no DML triggers (only DDL). Always wrap in an explicit transaction if unsure — TRUNCATE can be rolled back if part of an open transaction.

7.6 MERGE (Upsert)

```

MERGE dbo.Products AS target
USING (SELECT @Sku AS Sku, @Name AS Name, @Price AS Price) AS source
ON target.SKU = source.SKU
WHEN MATCHED THEN
    UPDATE SET Name = source.Name, Price = source.Price
WHEN NOT MATCHED BY TARGET THEN
    INSERT (SKU, Name, Price) VALUES (source.Sku, source.Name, source.Price)
WHEN NOT MATCHED BY SOURCE THEN
    DELETE;
GO

-- Tip: Use separate IF EXISTS ... UPDATE ELSE INSERT in production
-- MERGE has known concurrency issues; Microsoft recommends caution.

```

8. SQL Clauses

Clauses are the building blocks of a SELECT statement. They execute in a specific logical order (different from written order) which determines how aliases, aggregates, and filters interact.

8.1 Logical Execution Order

Step	What Happens
1. FROM / JOIN	Build the source rowset

Step	What Happens
2. WHERE	Row filter
3. GROUP BY	Aggregate groups
4. HAVING	Filter on aggregates
5. SELECT	Project columns; evaluate expressions
6. DISTINCT	Deduplicate
7. ORDER BY	Sort
8. OFFSET / FETCH	Paging

Why aliases cannot be in WHERE: Because WHERE runs BEFORE SELECT. The expression `SELECT Price\*1.1 AS NewPrice FROM Products WHERE NewPrice > 100` fails — NewPrice does not exist yet. Use `WHERE Price\*1.1 > 100` or repeat the expression. ORDER BY and HAVING CAN use SELECT aliases because they run after SELECT (or after GROUP BY in HAVING's case for aggregate aliases in some engines).

8.2 WHERE Operators

- **Comparison** — =, <>, !=, <, <=, >, >=, BETWEEN x AND y, NOT BETWEEN.
- **Set** — IN (1,2,3), NOT IN (1,2,3). Beware NULL in NOT IN — returns no rows.
- **Pattern** — LIKE with % (any chars), _ (one char), [abc] (char class), [^abc] (not class).
- **NULL** — IS NULL, IS NOT NULL. NULL = NULL returns UNKNOWN (not TRUE).
- **Logical** — AND, OR, NOT. Use parentheses — AND binds tighter than OR.
- **EXISTS** — Subquery existence check. Often faster than IN for correlated subqueries.
- **ALL / ANY / SOME** — Compare against a subquery result set. Rare but useful.

8.3 LIKE Patterns

```
-- Starts with "abc"
WHERE Name LIKE 'abc%'

-- Ends with "xyz"
WHERE Name LIKE '%xyz'

-- Contains "demo"
WHERE Name LIKE '%demo%'

-- Exactly 4 chars, 2nd char is "a"
WHERE Code LIKE '_a__'

-- Starts with letter a-m
WHERE Name LIKE '[a-m]%'

-- Does NOT start with a digit
WHERE Code LIKE '[^0-9]%'

-- ▲ Leading wildcards prevent index usage; use Full-Text Search instead.
```

8.4 ORDER BY

- **Default** — ASCENDING. Use DESC for descending.
- **Multiple columns** — `ORDER BY Country ASC, Name DESC` — primary then secondary.
- **COLLATE** — Override sort order: `ORDER BY Name COLLATE Latin1\_General\_CI\_AS`.
- **TOP with ORDER BY** — TOP without ORDER BY is non-deterministic. Always specify.
- **NULLS first/last** — SQL Server sorts NULLs first in ASC, last in DESC (no native NULLS FIRST clause — use CASE WHEN IS NULL trick).

8.5 OFFSET / FETCH (Modern Pagination)

```
-- Page 3 of 20 rows per page (skip 40, take 20)
SELECT CustomerID, Name
FROM dbo.Customers
ORDER BY Name
OFFSET 40 ROWS
FETCH NEXT 20 ROWS ONLY;

-- Top N (no skip)
SELECT TOP (20) CustomerID, Name FROM dbo.Customers ORDER BY Name;
-- Equivalent to: OFFSET 0 ROWS FETCH NEXT 20 ROWS ONLY
```

9. Joins

Joins combine rows from two or more tables based on a related column. They are the most-used feature in T-SQL after WHERE. Knowing the join types and their performance characteristics is essential.

9.1 Join Types

Type	Result	Use Case
INNER JOIN	Matched rows in both tables	Most common
LEFT [OUTER] JOIN	All left rows; matched right rows or NULLs	Optional related data
RIGHT [OUTER] JOIN	All right rows; matched left rows or NULLs	Rare (rewrite as LEFT)
FULL [OUTER] JOIN	All rows from both; NULLs for non-matches	Reconciliation
CROSS JOIN	Cartesian product (every left × every right)	Generate combinations
CROSS APPLY / OUTER APPLY	Invoke table-valued expression per left row	Per-row subquery
Self-join	Join a table to itself with aliases	Hierarchies

9.2 INNER JOIN Example

```
SELECT o.OrderID, o.OrderDate, c.Name AS Customer, p.Name AS Product,
       oi.Quantity, oi.UnitPrice
FROM   dbo.Orders AS o
JOIN   dbo.Customers AS c ON c.CustomerID = o.CustomerID
JOIN   dbo.OrderItems AS oi ON oi.OrderID = o.OrderID
JOIN   dbo.Products AS p ON p.ProductID = oi.ProductID
WHERE  o.OrderDate >= '2026-01-01'
ORDER BY o.OrderDate DESC, o.OrderID;
```

9.3 LEFT JOIN with NULL Check

```
-- Find customers who have NOT placed any orders in the last 30 days
SELECT c.CustomerID, c.Name
FROM   dbo.Customers c
LEFT JOIN dbo.Orders o
       ON o.CustomerID = c.CustomerID
       AND o.OrderDate >= DATEADD(day, -30, SYSUTCDATETIME())
WHERE  o.OrderID IS NULL;

-- Δ Filter on the right table in the ON clause, NOT the WHERE –
-- WHERE would convert LEFT JOIN into INNER JOIN.
```

9.4 Self-Join (Hierarchy)

```
-- Employees and their managers
SELECT e.Name AS Employee, m.Name AS Manager
FROM   dbo.Employees e
LEFT JOIN dbo.Employees m ON m.EmployeeID = e.ManagerID;
```

9.5 CROSS APPLY (Per-Row TVF)

```
-- Top 3 recent orders per customer (cannot do with plain JOIN)
SELECT c.Name, o.OrderID, o.OrderDate, o.Total
FROM   dbo.Customers c
CROSS APPLY (
    SELECT TOP (3) OrderID, OrderDate, Total
    FROM   dbo.Orders
    WHERE  CustomerID = c.CustomerID
    ORDER BY OrderDate DESC
) o;
```

10. Set Operators

Set operators combine the results of two queries into a single result set. The two queries must return the same number of columns with compatible types. They are useful for combining similar data from different sources or comparing rowsets.

10.1 The Four Operators

Operator	Behaviour	Notes
UNION	Combine, remove duplicates	Slow on large inputs (sort/hash for dedup)

Operator	Behaviour	Notes
UNION ALL	Combine, keep duplicates	Fastest; usually what you want
INTERSECT	Rows present in both	Deduplicates result
EXCEPT	Rows in first, not in second	Useful for "missing" data checks

10.2 Examples

```
-- Combine active and archived customers (no duplicates)
SELECT Name, Email FROM dbo.Customers WHERE IsActive = 1
UNION
SELECT Name, Email FROM dbo.Customers_Archive;

-- Combine with duplicates preserved (faster)
SELECT Name, Email FROM dbo.Customers WHERE IsActive = 1
UNION ALL
SELECT Name, Email FROM dbo.Customers_Archive;

-- Customers who have placed an order in 2025 AND 2026
SELECT CustomerID FROM dbo.Orders WHERE YEAR(OrderDate) = 2025
INTERSECT
SELECT CustomerID FROM dbo.Orders WHERE YEAR(OrderDate) = 2026;

-- Customers who have NOT placed an order in 2026
SELECT CustomerID FROM dbo.Customers
EXCEPT
SELECT CustomerID FROM dbo.Orders WHERE YEAR(OrderDate) = 2026;
```

10.3 Rules

- **Same column count** — Both queries must return the same number of columns.
- **Compatible types** — Column types must be implicitly convertible. The result type is based on precedence rules.
- **Column names** — Result uses the first query's column names. Use aliases there.
- **ORDER BY** — Only one ORDER BY at the end, applies to the combined result.
- **NULL handling** — Set operators treat NULLs as equal (unlike =). EXCEPT and INTERSECT treat NULL and NULL as matching.

11. Functions (Built-in)

T-SQL ships with hundreds of built-in functions. They are non-deterministic (may return different values on each call, like GETDATE) or deterministic (same inputs always yield same output, like UPPER). Knowing the common categories saves you from reinventing logic in procedural code.

11.1 Function Categories

Category	Description	Examples
Aggregate	Operate on a set, return scalar	SUM, AVG, MIN, MAX, COUNT
Window	Aggregate over a window of rows	ROW_NUMBER, RANK, LEAD, SUM OVER
Scalar	One value in, one out	UPPER, ROUND, ISNULL
Rowset	Return a table	OPENJSON, OPENXML, CONTAINSTABLE
Ranking	Per-row rank in a result set	ROW_NUMBER, RANK, DENSE_RANK, NTILE

11.2 NULL-Handling Functions (Critical)

```
-- ISNULL: replace NULL with a value (SQL Server specific)
SELECT ISNULL(Email, N'(no email)') FROM dbo.Customers;

-- COALESCE: ANSI standard, supports multiple args, returns first non-NULL
SELECT COALESCE(WorkPhone, MobilePhone, HomePhone, N'(no phone)')
FROM dbo.Contacts;

-- NULLIF: return NULL if two args equal (e.g., avoid divide by zero)
SELECT Sales / NULLIF(Units, 0) AS AvgPrice FROM dbo.DailySales;

-- ▲ ISNULL vs COALESCE:
-- ISNULL takes 2 args, returns type of first arg
-- COALESCE takes N args, returns type with highest precedence
-- COALESCE is ANSI – preferred for portability
```

11.3 Conversion Functions

```
-- CAST (ANSI, simple)
SELECT CAST('42' AS INT); -- 42
SELECT CAST(GetDate() AS DATE); -- date only

-- CONVERT (SQL Server, with style)
SELECT CONVERT(VARCHAR(10), GetDate(), 120); -- '2026-07-03' (ISO)
SELECT CONVERT(DECIMAL(10,2), '123.456'); -- 123.46 (rounded)

-- TRY_CAST / TRY_CONVERT (return NULL on failure)
SELECT TRY_CAST('abc' AS INT); -- NULL (no error)
SELECT TRY_CONVERT(DATE, '2026-13-45'); -- NULL

-- PARSE / TRY_PARSE (culture-aware, slower)
SELECT TRY_PARSE('01/02/2026' AS DATE USING 'en-US'); -- 2026-01-02
SELECT TRY_PARSE('01/02/2026' AS DATE USING 'en-GB'); -- 2026-02-01
```

11.4 System Functions

- **@@IDENTITY** — Last IDENTITY value in current session (any scope, includes triggers — can be wrong). Avoid.
- **SCOPE_IDENTITY()** — Last IDENTITY in current scope (safer than @@IDENTITY). Use after a single INSERT.

- **IDENT_CURRENT('table')** — Last IDENTITY for a specific table (any session/scope).
- **@@ROWCOUNT** — Number of rows affected by the last statement.
- **@@ERROR** — Error number of last statement (legacy — use TRY/CATCH + ERROR_*).
- **@@TRANCOUNT** — Number of active BEGIN TRANSACTIONS in the session.
- **ROWCOUNT_BIG()** — Like @@ROWCOUNT but BIGINT for > 2B rows.

12. String Functions

T-SQL has dozens of built-in string functions. The most-used are LEFT, RIGHT, SUBSTRING, LEN, REPLACE, CHARINDEX, STUFF, CONCAT, and STRING_SPLIT (2016+). Knowing them lets you do most string manipulation in SQL without round-tripping to the application.

12.1 Common String Functions

Function	Description	Returns
LEN(s)	Character count (excludes trailing spaces)	INT
DATALength(s)	Byte count (includes trailing spaces)	INT
LEFT(s, n) / RIGHT(s, n)	First / last n characters	VARCHAR/NVARCHAR
SUBSTRING(s, start, length)	1-indexed substring	Same as input
CHARINDEX(find, s [, start])	1-indexed position of find; 0 if not found	INT
PATINDEX('%pattern%', s)	Position of pattern with wildcards	INT
REPLACE(s, old, new)	Replace all occurrences	Same as input
STUFF(s, start, len, new)	Delete len chars at start, insert new	Same as input
UPPER(s) / LOWER(s)	Case conversion	Same as input
LTRIM(s) / RTRIM(s) / TRIM(s)	Remove leading / trailing / both spaces	Same as input
REPLICATE(s, n)	Repeat s n times	Same as input
REVERSE(s)	Reverse string	Same as input
CONCAT(a, b, c)	Concatenate, NULL → empty string	NVARCHAR(MAX)
CONCAT_WS(sep, a, b, c)	Concatenate with separator, skip NULLs	NVARCHAR(MAX)
FORMAT(value, format [, culture])	.NET-style formatting	NVARCHAR
STRING_SPLIT(s, sep)	Table of substrings (2016+)	Table
STRING_AGG(s, sep)	Concat rows into one string (2017+)	NVARCHAR(MAX)

12.2 Examples

```
-- Get initials from a full name
SELECT SUBSTRING(FirstName, 1, 1) + '. ' + SUBSTRING(LastName, 1, 1) + '.'
FROM dbo.Employees;

-- Format a phone number
SELECT STUFF(STUFF(PhoneNumber, 4, 0, '-'), 8, 0, '-')
FROM dbo.Contacts; -- 1234567890 → 123-456-7890

-- Split comma-separated tags
SELECT value AS Tag
FROM dbo.Articles
CROSS APPLY STRING_SPLIT(Tags, ',');

-- Aggregate names into a comma-separated list (2017+)
SELECT DepartmentID,
       STRING_AGG(Name, ', ') WITHIN GROUP (ORDER BY Name) AS Members
FROM dbo.Employees
GROUP BY DepartmentID;

-- Pattern matching with PATINDEX
SELECT * FROM dbo.Products
WHERE PATINDEX('%[0-9]%', Name) > 0; -- Name contains a digit
```

13. Date Functions

Date math is one of the most common query needs. T-SQL provides GETDATE / SYSDATETIME for current time, DATEADD / DATEDIFF for arithmetic, DATEPART / DATENAME for components, and several convenience functions like YEAR, MONTH, DAY.

13.1 Getting the Current Date/Time

Function	Returns	Notes
GETDATE()	DATETIME	Server-local time
GETUTCDATE()	DATETIME	UTC time
SYSDATETIME()	DATETIME2(7)	Higher precision, server-local
SYSUTCDATETIME()	DATETIME2(7)	Higher precision, UTC — recommended
SYSDATETIMEOFFSET()	DATETIMEOFFSET	UTC + offset

13.2 Date Arithmetic

```
-- DATEADD(unit, n, date): add n units to date
SELECT DATEADD(day, 30, OrderDate) AS DueDate FROM dbo.Orders;
SELECT DATEADD(hour, -2, SYSUTCDATETIME()); -- 2 hours ago

-- DATEDIFF(unit, start, end): integer difference
SELECT DATEDIFF(day, OrderDate, ShipDate) AS ShipDays
FROM dbo.Orders
WHERE ShipDate IS NOT NULL;
```

```
-- DATEDIFF_BIG: BIGINT for large ranges (ms across years)
SELECT DATEDIFF_BIG(millisecond, '2000-01-01', SYSUTCDATETIME());

-- DATEDIFF parts:
-- year, quarter, month, dayofyear, day, week, hour, minute, second,
-- millisecond, microsecond, nanosecond
```

13.3 Extracting Components

```
-- DATEPART(unit, date): integer part
SELECT DATEPART(year, OrderDate) AS Yr,
       DATEPART(month, OrderDate) AS Mo,
       DATEPART(weekday, OrderDate) AS DOW -- 1=Sunday (depends on @@DATEFIRST)
FROM dbo.Orders;

-- DATENAME(unit, date): string name (localised)
SELECT DATENAME(month, OrderDate) AS MonthName, -- 'July'
       DATENAME(weekday, OrderDate) AS DayName -- 'Thursday'
FROM dbo.Orders;

-- YEAR, MONTH, DAY: shortcut functions
SELECT YEAR(OrderDate), MONTH(OrderDate), DAY(OrderDate) FROM dbo.Orders;
```

13.4 Date Construction & Truncation

```
-- DATEFROMPARTS / DATETIME2FROMPARTS (SQL Server 2012+)
SELECT DATEFROMPARTS(2026, 7, 3); -- 2026-07-03
SELECT DATETIME2FROMPARTS(2026, 7, 3, 14, 30, 0, 0, 0); -- 2026-07-03 14:30:00
SELECT SMALLDATETIMEFROMPARTS(2026, 7, 3, 14, 30);

-- Truncate to start of day
SELECT CAST(OrderDate AS DATE) AS DayOnly FROM dbo.Orders;
SELECT CAST(SYSUTCDATETIME() AS DATE) AS Today;

-- Start of week / month / year (EOMONTH = end of month)
SELECT DATEADD(day, DATEDIFF(day, 0, OrderDate), 0) AS DayStart,
       DATEADD(month, DATEDIFF(month, 0, OrderDate), 0) AS MonthStart,
       EOMONTH(OrderDate) AS MonthEnd
FROM dbo.Orders;
```

Sargability tip: Wrapping a column in a function (e.g., WHERE YEAR(OrderDate) = 2026) prevents index use. Rewrite as a range: WHERE OrderDate >= '2026-01-01' AND OrderDate < '2027-01-01'. The optimizer can use the index on OrderDate for the range but not for the function.

14. Aggregate Functions

Aggregate functions operate on a set of rows and return a single value. They are used with GROUP BY to produce per-group summaries, or without GROUP BY to produce a single summary for the entire result set.

14.1 Built-in Aggregates

Function	Description	Returns
COUNT(*)	Number of rows (including NULLs)	BIGINT
COUNT(col)	Number of non-NULL values	BIGINT
COUNT(DISTINCT col)	Number of distinct non-NULL values	BIGINT
SUM(col)	Sum of non-NULL numeric values	Type of col (promoted)
AVG(col)	Average of non-NULL values = SUM/COUNT	Type of col (promoted)
MIN(col)	Minimum value (works on strings/dates too)	Type of col
MAX(col)	Maximum value	Type of col
STRING_AGG(col, sep)	Concatenate strings (2017+)	NVARCHAR(MAX)
APPROX_COUNT_DISTINCT(col)	HyperLogLog estimate (2019+)	BIGINT
CHECKSUM_AGG(col)	Checksum of values (change detection)	INT
GROUPING(col)	1 if NULL due to ROLLUP/CUBE, 0 otherwise	INT

14.2 NULL Behaviour (Critical)

- **COUNT(*)** — Counts all rows, including those with NULLs in every column.
- **COUNT(col)** — Counts only non-NULL values of col. Skips NULLs silently.
- **SUM / AVG** — Skip NULLs. SUM of all NULLs returns NULL, not 0. Use COALESCE (SUM(x), 0) for a guaranteed number.
- **MIN / MAX** — Skip NULLs. Return NULL if all values are NULL.
- **COUNT(DISTINCT col)** — Counts distinct non-NULL values.
- **Aggregates in subqueries** — A scalar aggregate with no rows returns NULL — not 0 or empty string. Common bug in IF checks.

14.3 Examples

```
-- Without GROUP BY: single summary row
SELECT COUNT(*) AS TotalOrders,
       COUNT(DISTINCT CustomerID) AS UniqueCustomers,
       SUM(Total) AS Revenue,
       AVG(Total) AS AvgOrder,
       MIN(OrderDate) AS FirstOrder,
       MAX(OrderDate) AS LastOrder
FROM dbo.Orders
WHERE OrderDate >= '2026-01-01';

-- With GROUP BY: per-group summary
```

```

SELECT CustomerID,
       COUNT(*) AS Orders,
       SUM(Total) AS Revenue,
       AVG(Total) AS AvgOrder
FROM dbo.Orders
WHERE OrderDate >= '2026-01-01'
GROUP BY CustomerID
HAVING COUNT(*) > 5
ORDER BY Revenue DESC;

```

14.4 DISTINCT Inside Aggregates

```

-- Average of DISTINCT prices (no duplicates)
SELECT AVG(DISTINCT Price) FROM dbo.Products;

-- Count of unique customers who ordered in 2026
SELECT COUNT(DISTINCT CustomerID) AS UniqueCustomers
FROM dbo.Orders
WHERE YEAR(OrderDate) = 2026;

-- For huge tables, APPROX_COUNT_DISTINCT is much faster (within ~2% error)
SELECT APPROX_COUNT_DISTINCT(UserID) AS ApproxUniqueUsers
FROM dbo.ClickStream;

```

15. GROUP BY and HAVING

GROUP BY partitions a result set into groups, and aggregate functions produce a summary row per group. HAVING filters groups (after aggregation), analogous to WHERE filtering rows (before aggregation). Mastering GROUP BY is fundamental to reporting queries.

15.1 Basic GROUP BY

```

-- Total revenue and order count per customer in 2026
SELECT c.CustomerID,
       c.Name,
       COUNT(*) AS OrderCount,
       SUM(o.Total) AS TotalRevenue,
       AVG(o.Total) AS AvgOrder,
       MAX(o.OrderDate) AS LastOrder
FROM dbo.Orders o
JOIN dbo.Customers c ON c.CustomerID = o.CustomerID
WHERE o.OrderDate >= '2026-01-01'
GROUP BY c.CustomerID, c.Name
ORDER BY TotalRevenue DESC;

```

15.2 HAVING (Filter on Aggregates)

```

-- Only customers with revenue > 10,000 AND 5+ orders
SELECT c.CustomerID, c.Name, COUNT(*) AS Orders, SUM(o.Total) AS Revenue
FROM dbo.Orders o
JOIN dbo.Customers c ON c.CustomerID = o.CustomerID
WHERE o.OrderDate >= '2026-01-01'
GROUP BY c.CustomerID, c.Name
HAVING SUM(o.Total) > 10000 AND COUNT(*) >= 5;

-- ⚠ HAVING cannot use SELECT aliases in SQL Server:

```



```
-- HAVING Revenue > 10000 -- invalid: cannot use alias
-- HAVING SUM(o.Total) > 10000 -- ✓ repeat the expression
```

15.3 GROUP BY with Multiple Columns

```
-- Sales by year, then by month, then by region
SELECT YEAR(OrderDate) AS Yr,
       MONTH(OrderDate) AS Mo,
       Region,
       COUNT(*) AS Orders,
       SUM(Total) AS Revenue
FROM dbo.Orders
GROUP BY YEAR(OrderDate), MONTH(OrderDate), Region
ORDER BY Yr, Mo, Region;
```

15.4 GROUPING SETS / ROLLUP / CUBE

These are shorthand for multiple GROUP BYs in a single query — they produce subtotal and grand total rows alongside the detail rows.

```
-- GROUPING SETS: explicit list of grouping combinations
SELECT YEAR(OrderDate) AS Yr, Region, SUM(Total) AS Revenue
FROM dbo.Orders
GROUP BY GROUPING SETS (
    (YEAR(OrderDate), Region), -- per year per region
    (YEAR(OrderDate)), -- per year (Region = NULL)
    () -- grand total (Yr = NULL, Region = NULL)
);

-- ROLLUP: hierarchical subtotals (year → region → grand total)
SELECT YEAR(OrderDate) AS Yr, Region, SUM(Total) AS Revenue
FROM dbo.Orders
GROUP BY ROLLUP (YEAR(OrderDate), Region);

-- CUBE: every combination of grouping columns
SELECT YEAR(OrderDate) AS Yr, Region, SUM(Total) AS Revenue
FROM dbo.Orders
GROUP BY CUBE (YEAR(OrderDate), Region);
```

15.5 GROUPING() Function

When ROLLUP/CUBE/GROUPING SETS produce NULL subtotals, you need to distinguish them from real NULL values in the data. The GROUPING() function returns 1 if the NULL is a subtotal marker, 0 otherwise.

```
SELECT CASE WHEN GROUPING(YEAR(OrderDate)) = 1
           THEN 'ALL YEARS' ELSE CAST(YEAR(OrderDate) AS VARCHAR) END AS Yr,
       CASE WHEN GROUPING(Region) = 1
           THEN 'ALL REGIONS' ELSE Region END AS Region,
       SUM(Total) AS Revenue
FROM dbo.Orders
GROUP BY ROLLUP (YEAR(OrderDate), Region);
```

16. Subqueries

A subquery is a `SELECT` inside another `SELECT`, `INSERT`, `UPDATE`, or `DELETE`. They come in several forms: scalar, multi-row, correlated, and derived tables. Modern T-SQL favours CTEs and `JOIN`s over deeply nested subqueries for readability.

16.1 Subquery Categories

Type	Returns	Use Case
Scalar	Returns 1 row, 1 column	Where a single value is expected
Multi-row	Returns N rows, 1 column	With <code>IN</code> / <code>ANY</code> / <code>ALL</code>
Multi-column	Returns N rows, M columns	With <code>EXISTS</code> / row comparators
Correlated	References the outer query	Per-row evaluation
Derived table	Subquery in <code>FROM</code> clause	When you need a temp rowset
Common Table Expression (CTE)	Named subquery in <code>WITH</code> clause	Readability, recursion

16.2 Scalar Subquery

```
-- Each customer with the average order total across all customers
SELECT CustomerID, Total,
       (SELECT AVG(Total) FROM dbo.Orders) AS OverallAvg,
       Total - (SELECT AVG(Total) FROM dbo.Orders) AS DiffFromAvg
FROM dbo.Orders
WHERE OrderDate = CAST(SYSUTCDATETIME() AS DATE);
```

16.3 IN / NOT IN (Multi-row)

```
-- Customers who placed at least one order in 2026
SELECT * FROM dbo.Customers
WHERE CustomerID IN (SELECT CustomerID FROM dbo.Orders
                    WHERE YEAR(OrderDate) = 2026);

-- Δ NOT IN with NULLs returns NO rows!
-- If the subquery returns any NULL, NOT IN evaluates to UNKNOWN for all rows.
-- Fix: filter NULLs out OR use NOT EXISTS:
SELECT * FROM dbo.Customers c
WHERE NOT EXISTS (SELECT 1 FROM dbo.Orders o
                 WHERE o.CustomerID = c.CustomerID
                   AND YEAR(o.OrderDate) = 2026);
```

16.4 Correlated Subquery

```
-- Each customer with their most recent order date
SELECT c.CustomerID, c.Name,
       (SELECT MAX(o.OrderDate)
        FROM dbo.Orders o
        WHERE o.CustomerID = c.CustomerID) AS LastOrder
```

```
FROM dbo.Customers c;
```

```
-- Correlated subqueries execute per outer row – can be slow.
-- Often rewriteable as JOIN with GROUP BY or as a window function:
```

16.5 EXISTS / NOT EXISTS

```
-- Customers who have at least one order with total > 1000
SELECT * FROM dbo.Customers c
WHERE EXISTS (SELECT 1 FROM dbo.Orders o
              WHERE o.CustomerID = c.CustomerID
                 AND o.Total > 1000);
```

```
-- EXISTS is usually faster than IN for correlated checks:
-- - Short-circuits at first match
-- - No NULL handling issues
-- - Optimizer can transform to a semi-join
```

16.6 Derived Table (Subquery in FROM)

```
-- Average order total per customer (then filter the averages)
SELECT CustomerID, AvgTotal
FROM (
    SELECT CustomerID, AVG(Total) AS AvgTotal
    FROM dbo.Orders
    GROUP BY CustomerID
) AS agg
WHERE AvgTotal > 500;

-- Modern alternative: CTE (cleaner, can be referenced multiple times)
WITH CustomerAvg AS (
    SELECT CustomerID, AVG(Total) AS AvgTotal
    FROM dbo.Orders
    GROUP BY CustomerID
)
SELECT * FROM CustomerAvg WHERE AvgTotal > 500;
```

17. Common Table Expressions (CTE)

A CTE is a named temporary result set defined with WITH, valid for the duration of a single SELECT/INSERT/UPDATE/DELETE/MERGE statement. CTEs improve readability of complex queries and enable recursive queries that walk hierarchies.

17.1 Basic (Non-recursive) CTE

```
WITH OrderStats AS (
    SELECT CustomerID,
           COUNT(*) AS Orders,
           SUM(Total) AS Revenue,
           AVG(Total) AS AvgOrder
    FROM dbo.Orders
    WHERE OrderDate >= '2026-01-01'
    GROUP BY CustomerID
)
SELECT c.Name, os.Orders, os.Revenue, os.AvgOrder
FROM dbo.Customers c
```

```
JOIN OrderStats os ON os.CustomerID = c.CustomerID
ORDER BY os.Revenue DESC;
```

17.2 Multiple CTEs

```
WITH ActiveCustomers AS (
    SELECT CustomerID, Name FROM dbo.Customers WHERE IsActive = 1
),
RecentOrders AS (
    SELECT CustomerID, COUNT(*) AS Orders
    FROM dbo.Orders
    WHERE OrderDate >= DATEADD(day, -30, SYSUTCDATETIME())
    GROUP BY CustomerID
)
SELECT ac.Name, COALESCE(ro.Orders, 0) AS Last30DayOrders
FROM ActiveCustomers ac
LEFT JOIN RecentOrders ro ON ro.CustomerID = ac.CustomerID
ORDER BY Last30DayOrders DESC;
```

17.3 Recursive CTE — Hierarchy Walk

```
-- Walk an employee → manager chain
WITH EmployeeHierarchy AS (
    -- Anchor: top-level employees (no manager)
    SELECT EmployeeID, Name, ManagerID, 1 AS Level,
           CAST(Name AS NVARCHAR(1000)) AS Path
    FROM dbo.Employees
    WHERE ManagerID IS NULL

    UNION ALL

    -- Recursive: employees whose manager is in the previous level
    SELECT e.EmployeeID, e.Name, e.ManagerID, h.Level + 1,
           CAST(h.Path + ' > ' + e.Name AS NVARCHAR(1000))
    FROM dbo.Employees e
    JOIN EmployeeHierarchy h ON e.ManagerID = h.EmployeeID
)
SELECT EmployeeID, Name, Level, Path
FROM EmployeeHierarchy
ORDER BY Path;

-- Limit recursion depth (default 100; raise if needed):
OPTION (MAXRECURSION 0); -- 0 = no limit (careful!)
```

17.4 Recursive CTE — Date Series

```
-- Generate a date series for a calendar report
WITH DateSeries AS (
    SELECT CAST('2026-01-01' AS DATE) AS Dt
    UNION ALL
    SELECT DATEADD(day, 1, Dt)
    FROM DateSeries
    WHERE Dt < '2026-12-31'
)
SELECT * FROM DateSeries OPTION (MAXRECURSION 366);
```

18. Temporary Tables

Temporary tables store intermediate results in tempdb. They come in two scoping flavours: local (#name, visible to current session) and global (##name, visible to all sessions). They are automatically dropped when the session disconnects, but explicit DROP is good practice.

18.1 Local vs Global Temp Tables

Type	Visibility	Cleanup	Use Case
#Local	Current session only	On session end	Most common
##Global	All sessions (same instance)	When last session using it ends	Rare; risky
Tempdb.dbo.TableName	Manual creation in tempdb	Manual DROP	When you need tempdb permanence

18.2 Creating and Using

```
-- Create with explicit schema
CREATE TABLE #TopCustomers (
    CustomerID INT PRIMARY KEY,
    Name NVARCHAR(100),
    Revenue DECIMAL(18,2)
);

-- Populate via INSERT...SELECT
INSERT INTO #TopCustomers (CustomerID, Name, Revenue)
SELECT c.CustomerID, c.Name, SUM(o.Total)
FROM dbo.Orders o
JOIN dbo.Customers c ON c.CustomerID = o.CustomerID
WHERE o.OrderDate >= '2026-01-01'
GROUP BY c.CustomerID, c.Name
HAVING SUM(o.Total) > 10000;

-- Use it
SELECT * FROM #TopCustomers ORDER BY Revenue DESC;

-- Clean up
DROP TABLE #TopCustomers;

-- Create with SELECT INTO (faster, no schema)
SELECT c.CustomerID, c.Name, SUM(o.Total) AS Revenue
INTO #TopCustomers2
FROM dbo.Orders o
JOIN dbo.Customers c ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.Name;
```

18.3 Indexes on Temp Tables

- **CREATE INDEX after data load** — Loading data into an indexed table is slower than loading into a heap then adding the index. For large temp tables, do INSERT first, CREATE INDEX second.
- **Primary key / unique constraint** — Allowed; create clustered on the most selective column.

- **Statistics** — Auto-created on temp tables — same rules as user tables. May be stale if data volume changes drastically.
- **Recompiles** — Temp tables cause stored proc recompiles more often than table variables (which have no cardinality). Useful when stale stats cause bad plans.

18.4 When to Use Temp Tables

- **Large intermediate result** — When the rowset is > 10,000 rows and reused in multiple operations.
- **Complex multi-step transformations** — Easier to read than nested CTEs.
- **Need indexes for joins** — Temp tables support indexes; table variables do not (before SQL Server 2019).
- **Cursored operations** — Staging data for a cursor or batch loop.

19. Table Variables

Table variables (@t) are declared like local variables but hold a rowset. They behave like temp tables for many use cases but have important differences in scoping, statistics, and transaction behaviour.

19.1 Declaration and Use

```
DECLARE @TopCustomers TABLE (
    CustomerID INT PRIMARY KEY,
    Name NVARCHAR(100),
    Revenue DECIMAL(18,2)
);

INSERT INTO @TopCustomers (CustomerID, Name, Revenue)
SELECT c.CustomerID, c.Name, SUM(o.Total)
FROM dbo.Orders o
JOIN dbo.Customers c ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.Name
HAVING SUM(o.Total) > 10000;

SELECT * FROM @TopCustomers ORDER BY Revenue DESC;
-- Variable dropped automatically when batch ends
```

19.2 Table Variable vs Temp Table

Property	Table Variable (@t)	Temp Table (#t)
Scope	Batch / proc	Session
Storage	tempdb (large) / memory (small)	tempdb
Statistics	None (estimated 1 row!)	Auto-created
Indexes	Only declared constraints (PK/UNIQUE)	Any indexes
Schema changes	Not allowed after DECLARE	ALTER TABLE works
Transaction rollback	NOT rolled back	Rolled back

Property	Table Variable (@t)	Temp Table (#t)
Recompiles	Fewer (no stats trigger)	More (stats changes)
Best for	Small rowset, loop accumulator	Large intermediate result

19.3 Table-Valued Parameters (TVPs)

```
-- Define a table type once
CREATE TYPE dbo.IntList AS TABLE (Value INT NOT NULL PRIMARY KEY);
GO

-- Use as a stored proc parameter
CREATE PROCEDURE dbo.GetOrdersForCustomers
    @CustomerIDs dbo.IntList READONLY
AS
BEGIN
    SELECT o.* FROM dbo.Orders o
    JOIN @CustomerIDs c ON c.Value = o.CustomerID;
END;
GO

-- Call from app (pass DataTable from .NET)
DECLARE @List dbo.IntList;
INSERT INTO @List VALUES (1), (2), (3);
EXEC dbo.GetOrdersForCustomers @List;
```

SQL Server 2019 change: Table variables now get cardinality estimation from a hidden tempdb statistic (deferred compile), eliminating the old "1-row estimate" problem. This makes table variables much safer for moderate-size data than in earlier versions. Still prefer temp tables for large data with complex joins where statistics matter.

20. Views

A view is a virtual table whose contents are defined by a stored SELECT. Views do not store data (except indexed views) — they are expanded inline at query time. They simplify complex queries, provide a security boundary, and abstract schema changes from consumers.

20.1 Creating a View

```
CREATE VIEW Sales.Vw_CustomerRevenue
WITH SCHEMABINDING
AS
    SELECT c.CustomerID, c.Name, c.Country,
           COUNT_BIG(*) AS OrderCount,
           SUM(o.Total) AS TotalRevenue,
           MAX(o.OrderDate) AS LastOrder
    FROM dbo.Customers c
    JOIN dbo.Orders o ON o.CustomerID = c.CustomerID
    GROUP BY c.CustomerID, c.Name, c.Country;
GO

-- Use it like a table
SELECT * FROM Sales.Vw_CustomerRevenue
```

```
WHERE Country = 'US' AND TotalRevenue > 10000
ORDER BY TotalRevenue DESC;
```

20.2 SCHEMABINDING

- **What it does** — Binds the view to the underlying schema — prevents ALTER/DROP on dependent tables/columns until the view is dropped.
- **Why use it** — Protects against accidental schema breakage. Required for indexed views.
- **Syntax rules** — Must use two-part names (schema.table), cannot use SELECT *, cannot reference other views (only tables).

20.3 Indexed Views (Materialised Views)

An indexed view stores the result physically — much faster reads, slower writes. First create a unique clustered index on the view, then optionally nonclustered indexes. The view definition must be deterministic and use SCHEMABINDING.

```
-- Already created with SCHEMABINDING above; now index it
CREATE UNIQUE CLUSTERED INDEX IX_Vw_CustomerRevenue_Cust
  ON Sales.Vw_CustomerRevenue (CustomerID);
GO

-- Now SELECT * FROM Sales.Vw_CustomerRevenue reads from the index, not the
-- underlying tables. Writes to Customers/Orders propagate to the view index.

-- ▲ Enterprise edition can use the view index even when the query does not
-- reference the view directly (view-matching). Standard edition requires
-- NOEXPAND hint.
```

20.4 View Limitations

- **Single SELECT** — A view is one SELECT statement — no procedural code.
- **No ORDER BY** — A view cannot have ORDER BY unless TOP/OFFSET is also specified. Ordering is the consumer's responsibility.
- **Limited DML through views** — UPDATE through a view works only if the view references one base table and does not compute derived columns.
- **Indexed view restrictions** — No OUTER JOINs, no COUNT(*) (use COUNT_BIG), no non-deterministic functions, no subqueries.

21. Indexes

Indexes are the most powerful performance tool in SQL Server. They are B-tree structures that let the engine find rows without scanning the entire table. The right indexes can cut query time by 100x; the wrong ones slow down writes and bloat storage. Index design is a core DBA and developer skill.

21.1 Clustered vs Nonclustered

Type	Limit	Physical Order	Best For
Clustered	1 per table	Yes — defines physical order of data pages	PK (usually)
Nonclustered	Up to 999 (per table)	No — separate B-tree with row locator	Search columns, FK columns
Unique	Both	Enforces uniqueness	Alternate keys
Filtered	Nonclustered	Index only rows matching a WHERE predicate	Sparse columns
Columnstore	Both	Column-oriented, compressed	Analytics
In-memory	Both	Memory-optimised, no disk	In-Memory OLTP

21.2 Creating Indexes

```
-- Clustered index on PK (created implicitly by PRIMARY KEY constraint)
CREATE TABLE dbo.Products (
    ProductID INT IDENTITY PRIMARY KEY, -- creates clustered PK by default
    SKU VARCHAR(20) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    CategoryID INT NOT NULL,
    Price DECIMAL(10,2) NOT NULL
);

-- Nonclustered index on SKU for lookup queries
CREATE UNIQUE NONCLUSTERED INDEX IX_Products_SKU ON dbo.Products (SKU);

-- Composite nonclustered index for category + price queries
CREATE NONCLUSTERED INDEX IX_Products_Category_Price
    ON dbo.Products (CategoryID, Price DESC);

-- Covering index: include columns to avoid key lookups
CREATE NONCLUSTERED INDEX IX_Products_Category_Include
    ON dbo.Products (CategoryID)
    INCLUDE (Name, Price); -- not key, but stored in leaf for coverage

-- Filtered index: only active products
CREATE NONCLUSTERED INDEX IX_Products_Active
    ON dbo.Products (CategoryID)
    WHERE IsActive = 1;
```

21.3 INCLUDE vs Key Columns

- **Key columns** — Indexed columns used in WHERE, JOIN, ORDER BY, GROUP BY. They are at every level of the B-tree.
- **Included columns** — Stored only at the leaf level. Cannot be used for seeks or sorts. They make the index "covering" for SELECT queries — no need to lookup the base table.
- **Trade-off** — More included columns = larger index = slower writes but faster covering reads. Pick the columns the query selects.

21.4 How Indexes Are Used

- **Seek** — B-tree traversal using the leading column(s). Fastest path — $O(\log N)$.
- **Scan** — Read entire index leaf level. Slower but cheaper than scanning the clustered index for narrow nonclustered indexes.
- **Key lookup** — After a nonclustered seek, fetch remaining columns from the clustered index. One lookup per row — can be catastrophic at scale.
- **Covering index** — Includes all needed columns → no lookup, single index read.

Leading column rule: A composite index on (CategoryID, Price) can seek on CategoryID, or CategoryID AND Price — but NOT on Price alone. The leading column must be in the WHERE clause. Order columns by selectivity and access pattern. If you need both (Price) alone and (CategoryID, Price), create two separate indexes.

21.5 Index Maintenance

```
-- Check fragmentation
SELECT OBJECT_NAME(ips.object_id) AS TableName,
       i.name AS IndexName,
       ips.avg_fragmentation_in_percent
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ips
JOIN sys.indexes i ON i.object_id = ips.object_id AND i.index_id = ips.index_id
WHERE ips.avg_fragmentation_in_percent > 5
ORDER BY ips.avg_fragmentation_in_percent DESC;

-- Maintenance rules of thumb:
-- < 5% fragmentation → leave alone
-- 5-30% fragmentation → REORGANIZE (online, lightweight)
-- > 30% fragmentation → REBUILD (offline by default; ONLINE = Enterprise only)
ALTER INDEX IX_Products_SKU ON dbo.Products REORGANIZE;
ALTER INDEX ALL ON dbo.Products REBUILD WITH (ONLINE = ON); -- Enterprise feature
```

22. Stored Procedures

A stored procedure is a precompiled, named T-SQL block stored in the database. They encapsulate logic, centralise security (grant EXECUTE instead of table permissions), reduce network traffic, and benefit from plan caching.

22.1 Anatomy

```
CREATE OR ALTER PROCEDURE dbo.GetCustomerOrders
    @CustomerID INT,
    @StartDate DATE = NULL,
    @PageSize INT = 20,
    @PageNumber INT = 1
AS
BEGIN
    SET NOCOUNT ON; -- suppress "rows affected" messages (perf)
    SET XACT_ABORT ON; -- auto-rollback on error

    IF @StartDate IS NULL
        SET @StartDate = DATEADD(year, -1, SYSUTCDATETIME());
```

```

SELECT o.OrderID, o.OrderDate, o.Total,
       p.Name AS ProductName, oi.Quantity
FROM   dbo.Orders o
JOIN   dbo.OrderItems oi ON oi.OrderID = o.OrderID
JOIN   dbo.Products p ON p.ProductID = oi.ProductID
WHERE  o.CustomerID = @CustomerID
       AND o.OrderDate >= @StartDate
ORDER BY o.OrderDate DESC
OFFSET (@PageNumber - 1) * @PageSize ROWS
FETCH NEXT @PageSize ROWS ONLY;
END;
GO

EXEC dbo.GetCustomerOrders @CustomerID = 42, @PageNumber = 2;

```

22.2 Why Use Stored Procedures

- **Plan caching** — First execution compiles a plan; subsequent calls reuse it. Avoids recompilation cost on every call.
- **Network efficiency** — One RPC call sends parameters; client does not transmit SQL text. Less chatty.
- **Security** — Grant EXECUTE on the proc; users need no direct table access. Prevents SQL injection if you use parameters (never concatenate).
- **Encapsulation** — Schema changes inside the proc do not affect callers — contract is the parameter list and result shape.
- **Centralised business logic** — Multiple apps (.NET, Java, Python) call the same proc — logic lives in one place.

22.3 Parameter Sniffing

When SQL Server compiles a stored proc, it "sniffs" the first call's parameter values and builds a plan optimised for them. Subsequent calls reuse that plan even if their parameters would benefit from a different plan. This is "parameter sniffing" — usually good, sometimes bad.

```

-- Problem: first call with @Status='Active' (1M rows) builds a plan using
-- a scan. Subsequent call with @Status='Pending' (5 rows) reuses the scan
-- instead of a seek – orders of magnitude slower.

-- Fix 1: RECOMPILE hint for volatile procs
SELECT * FROM dbo.Orders WHERE Status = @Status OPTION (RECOMPILE);

-- Fix 2: OPTIMIZE FOR UNKNOWN (use density vector, not parameter value)
OPTION (OPTIMIZE FOR UNKNOWN);

-- Fix 3: local variable (defeats sniffing)
DECLARE @LocalStatus VARCHAR(10) = @Status;
SELECT * FROM dbo.Orders WHERE Status = @LocalStatus;

-- Fix 4 (best for SQL 2022+): Parameter Sensitive Plan Optimization (PSP)
-- automatically maintains multiple plans per parameter value.

```

22.4 OUTPUT Parameters

```
CREATE OR ALTER PROCEDURE dbo.InsertCustomer
    @Name NVARCHAR(100),
    @Email NVARCHAR(255),
    @NewID INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO dbo.Customers (Name, Email)
    VALUES (@Name, @Email);

    SET @NewID = SCOPE_IDENTITY();
END;
GO

DECLARE @ID INT;
EXEC dbo.InsertCustomer N'Acme', N'sales@acme.com', @ID OUTPUT;
SELECT @ID AS NewCustomerID;
```

23. User Defined Functions (UDF)

UDFs are T-SQL routines that return a scalar value or a table. They are callable from SELECT, WHERE, computed columns, and CHECK constraints. Historically a performance problem; SQL Server 2019+ added "scalar inlining" that fixes most issues for scalar UDFs.

23.1 UDF Types

Type	Returns	Usage	Implementation
Scalar	Returns one value	SELECT dbo.MyFunc(x)	Inline (2019+) or T-SQL
Inline TVF (iTVF)	Returns table, single SELECT	SELECT * FROM dbo.MyFunc(x)	Inline, fast
Multi-statement TVF (mTVF)	Returns table, multi-statement body	Same as iTVF	Slow; parallelism blocked

23.2 Scalar UDF (and Inlining)

```
-- Classic scalar UDF (pre-2019: row-by-row execution, no parallelism)
CREATE FUNCTION dbo.CalculateTax(@Amount DECIMAL(10,2), @State CHAR(2))
RETURNS DECIMAL(10,2)
AS
BEGIN
    DECLARE @Rate DECIMAL(5,4) =
        CASE @State
            WHEN 'CA' THEN 0.0875
            WHEN 'NY' THEN 0.08
            ELSE 0.06
        END;
    RETURN @Amount * @Rate;
END;
GO
```

```
-- SQL Server 2019+ auto-inlines this if the function is:
-- - Not recursive
-- - Single RETURN statement (after CASE/simplification)
-- - No UDF-specific restrictions (no table access, etc.)

-- Use:
SELECT OrderID, Total, dbo.CalculateTax(Total, State) AS Tax
FROM dbo.Orders;
```

23.3 Inline TVF (Preferred for Table-Returning)

```
CREATE FUNCTION dbo.GetRecentOrders(@CustomerID INT, @Days INT)
RETURNS TABLE
AS
RETURN (
    SELECT OrderID, OrderDate, Total
    FROM dbo.Orders
    WHERE CustomerID = @CustomerID
        AND OrderDate >= DATEADD(day, -@Days, SYSUTCDATETIME())
);
GO

-- Use:
SELECT * FROM dbo.GetRecentOrders(42, 30);
```

23.4 Multi-Statement TVF (Avoid if Possible)

```
CREATE FUNCTION dbo.GetTopProductsByCategory(@CategoryID INT)
RETURNS @Result TABLE (ProductID INT, Name NVARCHAR(200), Revenue DECIMAL(18,2))
AS
BEGIN
    INSERT INTO @Result
    SELECT p.ProductID, p.Name, SUM(oi.Quantity * oi.UnitPrice) AS Revenue
    FROM dbo.Products p
    JOIN dbo.OrderItems oi ON oi.ProductID = p.ProductID
    WHERE p.CategoryID = @CategoryID
    GROUP BY p.ProductID, p.Name;

    -- Could add more statements here...
    UPDATE @Result SET Name = UPPER(Name) WHERE Revenue > 1000;

    RETURN;
END;
GO

-- ⚠ mTVFs block parallelism and have no statistics; rewrite as iTVF whenever possible.
```

24. Triggers

A trigger is a special stored procedure that fires automatically in response to a DML (INSERT/UPDATE/DELETE) or DDL event. They enable auditing, cascading changes, and complex integrity checks. Use sparingly — they are easy to forget, hard to debug, and a common source of mysterious performance issues.

24.1 DML Trigger Types

Type	Timing	Use Case
AFTER (FOR)	After the action completes and is logged	Auditing, cascades
INSTEAD OF	Replaces the action; original never executes	Update through views
FOR INSERT/UPDATE/DELETE	Synonym for AFTER	Same as AFTER

24.2 inserted and deleted Pseudo-Tables

- **INSERT** — `inserted` contains the new rows; `deleted` is empty.
- **DELETE** — `deleted` contains the deleted rows; `inserted` is empty.
- **UPDATE** — Both: `deleted` has pre-update rows, `inserted` has post-update rows.
- **Multi-row** — A trigger fires ONCE per statement, not per row. The pseudo-tables may contain many rows — always write set-based trigger code.

24.3 AFTER Trigger Example (Audit Log)

```
CREATE OR ALTER TRIGGER dbo.tr_Customers_Audit
ON dbo.Customers
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    -- Audit INSERTs and UPDATEs
    INSERT INTO Audit.CustomerChanges
        (Action, CustomerID, OldName, NewName, OldEmail, NewEmail, ChangedBy,
        ChangedAt)
    SELECT
        CASE WHEN d.CustomerID IS NULL THEN 'INSERT' ELSE 'UPDATE' END,
        i.CustomerID, d.Name, i.Name, d.Email, i.Email,
        SUSER_SNAME(), SYSUTCDATETIME()
    FROM inserted i
    LEFT JOIN deleted d ON d.CustomerID = i.CustomerID;

    -- Audit DELETEs
    INSERT INTO Audit.CustomerChanges
        (Action, CustomerID, OldName, NewName, OldEmail, NewEmail, ChangedBy,
        ChangedAt)
    SELECT 'DELETE', d.CustomerID, d.Name, NULL, d.Email, NULL,
        SUSER_SNAME(), SYSUTCDATETIME()
    FROM deleted d
    WHERE d.CustomerID NOT IN (SELECT CustomerID FROM inserted);
END;
```

24.4 INSTEAD OF Trigger (Update through View)

```
CREATE VIEW dbo.Vw_CustomerWithStats AS
    SELECT c.*, (SELECT COUNT(*) FROM dbo.Orders o WHERE o.CustomerID = c.CustomerID)
    AS OrderCount
    FROM dbo.Customers c;
```

```
GO
```

```
-- UPDATE on this view fails because of the subquery. Add INSTEAD OF trigger:
CREATE TRIGGER dbo.tr_Vw_Customer_Update
ON dbo.Vw_CustomerWithStats
INSTEAD OF UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE c
    SET c.Name = i.Name,
        c.Email = i.Email
    FROM dbo.Customers c
    JOIN inserted i ON i.CustomerID = c.CustomerID;
END;
```

24.5 Trigger Pitfalls

- **Hidden logic** — A developer reading the proc does not see the trigger — mysterious side effects. Document triggers in code reviews.
- **Recursive triggers** — A trigger on table A inserts into table B whose trigger updates A — recursion. Disabled by default; enable with `RECURSIVE_TRIGGERS ON`.
- **Nested triggers** — Trigger B fires from within trigger A. Default max nesting is 32 — exceeded raises error and rolls back.
- **Performance** — Triggers add to every DML. Heavy logic in a trigger slows the calling statement, including bulk loads.
- **Set-based code** — A trigger fires once per statement. Use set-based `INSERT...SELECT` from inserted/deleted, NOT a cursor.

25. Transactions

A transaction is a unit of work that is atomic, consistent, isolated, and durable (ACID). SQL Server supports explicit transactions (`BEGIN TRAN / COMMIT / ROLLBACK`), implicit transactions (`SET IMPLICIT_TRANSACTIONS ON`), and autocommit (default — each statement is its own transaction).

25.1 Explicit Transactions

```
BEGIN TRY
    BEGIN TRANSACTION;

    INSERT INTO dbo.Orders (CustomerID, OrderDate, Total)
    VALUES (42, SYSUTCDATETIME(), 100.00);
    DECLARE @OrderID INT = SCOPE_IDENTITY();

    INSERT INTO dbo.OrderItems (OrderID, ProductID, Quantity, UnitPrice)
    SELECT @OrderID, ProductID, Qty, Price
    FROM @CartItems; -- table variable passed in

    UPDATE dbo.Inventory
    SET StockOnHand -= Quantity
    FROM dbo.Inventory inv
    JOIN @CartItems ci ON ci.ProductID = inv.ProductID;
```

```

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        THROW; -- re-raise to caller
    END CATCH;

```

25.2 Transaction Modes

Mode	Behaviour	When
Autocommit (default)	Each statement is its own transaction	Simple scripts
Implicit	BEGIN TRAN auto-started on first DML; COMMIT/ROLLBACK to end	Rare; ANSI default
Explicit	Manual BEGIN TRAN/COMMIT/ROLLBACK	Multi-statement units of work
Batch-scoped	Transaction tied to MARS batch	MARS sessions only

25.3 SAVE TRAN (Partial Rollback)

```

BEGIN TRANSACTION;
    INSERT INTO dbo.Log (Msg) VALUES ('start');
    SAVE TRAN PreRisk; -- named savepoint

    BEGIN TRY
        EXEC dbo.RiskyOp;
    END TRY
    BEGIN CATCH
        ROLLBACK TRAN PreRisk; -- back to savepoint, transaction still active
        INSERT INTO dbo.Log (Msg) VALUES ('recovered');
    END CATCH
COMMIT TRANSACTION;

```

25.4 XACT_ABORT and Error Handling

- **SET XACT_ABORT ON** — Any run-time error automatically rolls back the entire transaction. Without it, a statement-level error leaves the transaction open — caller must explicitly rollback. Always ON in production.
- **@@TRANCOUNT** — Number of open transactions in the current session. > 0 means transaction active.
- **THROW vs RAISERROR** — THROW (2012+) re-raises the caught exception with original severity, line, and message. RAISERROR is older, more flexible for custom messages but less faithful. Use THROW in CATCH blocks.
- **Nested transactions** — SQL Server has NO true nested transactions — only the outermost COMMIT persists. Inner BEGIN TRAN just increments @@TRANCOUNT. A single ROLLBACK rolls back everything. Use savepoints for partial rollback.

26. ACID Properties

ACID — Atomicity, Consistency, Isolation, Durability — defines the guarantees a transactional database makes. SQL Server enforces all four through the transaction manager, lock manager, and write-ahead log.

26.1 The Four Properties

Property	Guarantee	Mechanism
Atomicity	All-or-nothing: either every change in the transaction persists or none do	Transaction Manager
Consistency	Transaction moves the database from one valid state to another (constraints, triggers)	Constraints, Triggers
Isolation	Concurrent transactions appear to execute serially	Locks + Isolation Level
Durability	Once committed, changes survive crashes	Write-ahead log + buffer flush

26.2 Atomicity in Detail

A transaction is atomic — either all its operations commit together, or none do. If a transaction has 5 operations and the 4th fails, the first 3 are rolled back. This is implemented via the transaction log: on rollback, SQL Server reads the undo portion of log records and reverses each operation. Atomicity lets application code treat a multi-statement unit of work as a single operation.

26.3 Consistency in Detail

A transaction brings the database from one consistent state to another. Consistency is enforced by all constraints (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK), triggers, and application logic. If a transaction violates a constraint, it fails and is rolled back — the database remains in the prior consistent state. Consistency is a shared responsibility: the database enforces structural rules; the application enforces business rules that cannot be expressed as constraints.

26.4 Isolation in Detail

Isolation means concurrent transactions cannot see each other's uncommitted changes — they appear to run serially. Full isolation (SERIALIZABLE) is expensive: locks block other transactions, reducing concurrency. SQL Server offers a spectrum of isolation levels that trade off consistency for concurrency (READ UNCOMMITTED → READ COMMITTED → REPEATABLE READ → SNAPSHOT → SERIALIZABLE). The default is READ COMMITTED.

26.5 Durability in Detail

Once a transaction commits, its changes are permanent — they survive crashes, power loss, and restarts. This is implemented via write-ahead logging: the log record is flushed to disk **BEFORE** the commit returns to the caller. On crash recovery, SQL Server replays the log: REDO committed transactions' changes, UNDO uncommitted ones. Delayed durability (an opt-in feature) trades durability for throughput by batching log flushes — useful for high-throughput ETL but risks losing committed transactions on crash.

27. Locks and Blocking

Locking is how SQL Server enforces isolation. Every read and write acquires a lock on a resource (row, page, table). When two transactions want incompatible locks on the same resource, one blocks waiting for the other. Locking is normal; blocking is normal; only excessive blocking is a problem.

27.1 Lock Modes

Mode	Purpose	Compatibility	Common Source
Shared (S)	Read	Compatible with other S	SELECT (default)
Update (U)	Intent to update	Compatible with S, not U	UPDATE search phase
Exclusive (X)	Write	Incompatible with everything	INSERT/UPDATE/DELETE
Intent (IS/IX)	Higher-level intent	Indicate lower-level locks held	Most operations
Schema (Sch-S / Sch-M)	Schema stability / modification	Sch-M blocks everything	Compile / DDL
Key-Range	Range of index keys	Prevent phantom inserts	SERIALIZABLE
Bulk Update (BU)	Bulk load	Only with TABLOCK	BULK INSERT

27.2 Lock Granularity

- **Row / key** — Single row in a heap or index. Most granular; default for SELECT/UPDATE.
- **Page** — 8 KB page. Acquired when many rows on the same page are affected.
- **Extent** — 8 pages = 64 KB. Rare; mostly internal.
- **Table** — Whole table. Acquired when locks exceed escalation threshold or with TABLOCK.
- **Database** — Whole database. Used for schema changes and instance-level ops.
- **Lock escalation** — When a transaction holds > 5,000 locks on one object, SQL Server escalates to a single table lock to save memory.

27.3 Blocking vs Deadlocking

Phenomenon	Cause	Resolution	Action
Blocking	Process A waits for a lock held by B	Resolves when B commits/rolls back	Normal; tune queries
Deadlocking	A waits for B AND B waits for A	Never resolves; SQL Server kills victim	Bug — fix query order

27.4 Investigating Blocking

```
-- Who is blocking whom? Run this when you suspect blocking.
SELECT er.session_id AS BlockedSPID,
       er.blocking_session_id AS BlockingSPID,
       er.wait_type,
       er.wait_time / 1000.0 AS WaitSeconds,
       SUBSTRING(st.text, (er.statement_start_offset/2)+1,
                 (CASE er.statement_end_offset
                    WHEN -1 THEN DATALENGTH(st.text)
                    ELSE er.statement_end_offset
                 END - er.statement_start_offset)/2 + 1) AS BlockedSQL
FROM sys.dm_exec_requests er
CROSS APPLY sys.dm_exec_sql_text(er.sql_handle) st
WHERE er.blocking_session_id <> 0;

-- Detailed blocking graph (walk the chain)
SELECT * FROM sys.dm_tran_locks
WHERE request_status = 'WAIT'
ORDER BY resource_associated_entity_id;
```

27.5 Reducing Blocking

- **Keep transactions short** — Shorter transactions = shorter lock hold = less blocking.
- **Use the lowest isolation level that works** — READ COMMITTED is the default; SNAPSHOT or READ COMMITTED SNAPSHOT avoids read/write blocking.
- **Proper indexing** — A SELECT without an index may scan the whole table, taking and holding many locks. An index seek takes one lock.
- **Avoid long-running cursors** — Each fetch holds locks.
- **Process in a consistent order** — Always update tables in the same order across transactions to avoid deadlocks.
- **Consider NOLOCK carefully** — READ UNCOMMITTED avoids S locks but reads dirty data. Acceptable for analytics, dangerous for OLTP.

28. Isolation Levels

Isolation levels control how much one transaction can see of another's uncommitted changes. Higher isolation = more consistency but more blocking; lower isolation = more concurrency but more anomalies. SQL Server supports 5 pessimistic (lock-based) and 2 optimistic (row-version) levels.

28.1 The Five Levels

Level	Dirty Read	Non-Repeatable	Phantom	Notes
READ UNCOMMITTED	Yes	Yes	Yes	Dirty read — no S lock taken (NOLOCK)
READ COMMITTED (default)	No	Yes	Yes	S locks released after read
REPEATABLE READ	No	No	Yes	S locks held until commit
SNAPSHOT	No	No	No (uses row versions)	Optimistic; readers don't block writers
SERIALIZABLE	No	No	No	Range locks prevent phantom reads

28.2 Setting Isolation Level

```
-- Session-level (persists for the session)
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- Query-level hint (single statement)
SELECT * FROM dbo.Orders WITH (NOLOCK) WHERE CustomerID = 42;
SELECT * FROM dbo.Orders WITH (READCOMMITTED) WHERE CustomerID = 42;

-- Enable READ_COMMITTED_SNAPSHOT (RCSI) at database level
-- Requires exclusive DB access:
ALTER DATABASE SalesDB SET READ_COMMITTED_SNAPSHOT ON WITH ROLLBACK IMMEDIATE;
ALTER DATABASE SalesDB SET ALLOW_SNAPSHOT_ISOLATION ON;
```

28.3 Read Anomalies (What Each Level Prevents)

- **Dirty read** — Transaction A reads uncommitted changes made by B. If B rolls back, A has read data that never existed.
- **Non-repeatable read** — A reads a row twice and gets different values because B modified it in between.
- **Phantom read** — A runs the same query twice and gets different rows because B inserted or deleted some.

- **Lost update** — A reads, B reads, A writes, B writes — A's update is lost. REPEATABLE READ prevents; READ COMMITTED does not.

28.4 SNAPSHOT and RCSI (Optimistic Concurrency)

Snapshot isolation uses row versioning in tempdb: when a row is modified, the previous version is stored in the version store. Readers read the version that was current when their transaction started — no S locks, no blocking by writers. Writers still block writers (X locks).

- **SNAPSHOT** — Opt-in via SET TRANSACTION ISOLATION LEVEL SNAPSHOT. The whole transaction sees data as of its start. Requires ALLOW_SNAPSHOT_ISOLATION ON.
- **RCSI (READ COMMITTED SNAPSHOT)** — Replaces the default READ COMMITTED behaviour with row-version-based reads. No app code change — just enable. Recommended for most OLTP workloads; eliminates reader/writer blocking.
- **Version store overhead** — tempdb grows while long-running transactions need old versions. Monitor tempdb size and long-running queries.
- **Write conflicts** — Two SNAPSHOT transactions updating the same row: the second fails with "snapshot update conflict" — must retry.

29. Deadlocks

A deadlock occurs when two transactions each hold a lock the other needs, and neither can proceed. SQL Server detects the cycle and kills one transaction (the "victim") so the other can continue. Deadlocks are normal in busy systems but should be rare; frequent deadlocks indicate a design or query-order problem.

29.1 Classic Deadlock Scenario

```
-- Transaction A -- Transaction B
BEGIN TRAN; BEGIN TRAN;
UPDATE dbo.Accounts UPDATE dbo.Accounts
  SET Balance += 100 SET Balance += 50
  WHERE AccountID = 1; WHERE AccountID = 2;
                                -- holds X lock on row 2
-- holds X lock on row 1
UPDATE dbo.Accounts UPDATE dbo.Accounts
  SET Balance -= 50 SET Balance -= 100
  WHERE AccountID = 2; WHERE AccountID = 1;
                                -- waits for A's X lock on row 1
-- waits for B's X lock on row 2
-- DEADLOCK! SQL Server kills one (the victim) and rolls it back.
-- The other continues and commits.
COMMIT TRAN; COMMIT TRAN;
```

29.2 How SQL Server Detects Deadlocks

- **Lock monitor** — A background system process checks the wait-for graph every 5 seconds (more often if deadlocks are frequent).
- **Cycle detection** — If it finds a cycle (A waits for B, B waits for A), it selects a victim.

- **Victim selection** — Chooses the transaction whose rollback is cheapest (fewest log bytes). Override with SET DEADLOCK_PRIORITY LOW/NORMAL/HIGH.
- **Error 1205** — The victim receives error 1205: "Transaction was deadlocked on lock resources with another process and has been chosen as the deadlock victim."

29.3 Capturing Deadlocks

```
-- Modern way: Extended Events system_health session already captures deadlocks
-- Query the ring buffer:
SELECT XEvent.query('event_data') AS EventXML
FROM (
    SELECT CAST(target_data AS XML) AS TargetXML
    FROM sys.dm_xe_session_targets t
    JOIN sys.dm_xe_sessions s ON s.address = t.event_session_address
    WHERE s.name = 'system_health'
    AND t.target_name = 'ring_buffer'
) AS Data
CROSS APPLY TargetXML.nodes('//event[@name="xml_deadlock_report"]') AS XEvent(XEvent);

-- Or enable trace flags 1204 and 1222 to write deadlock graphs to the error log:
DBCC TRACEON (1204, 1222, -1); -- global, until restart
-- For persistent: add as startup parameters in SQL Server Configuration Manager.
```

29.4 Preventing Deadlocks

- **Consistent lock order** — Always access tables and rows in the SAME order across all transactions. This is the #1 fix. Code review for lock order.
- **Keep transactions short** — Shorter transactions = smaller window for deadlock.
- **Proper indexing** — A UPDATE without an index may scan and lock many rows, increasing deadlock surface. Add indexes to make UPDATE seek only the target.
- **Lower isolation level** — READ COMMITTED SNAPSHOT removes reader/writer deadlocks.
- **Use UPDLOCK for read-then-write** — Acquire update locks during the read phase so the write phase does not have to upgrade (which can deadlock).
- **Retry logic** — Catch error 1205 in application code and retry the transaction with a small delay. Essential for high-concurrency systems.

29.5 Application Retry Pattern

```
-- In T-SQL (simplified):
DECLARE @Attempts INT = 0, @MaxAttempts INT = 3;
WHILE @Attempts < @MaxAttempts
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;
        -- ... do work ...
        COMMIT TRANSACTION;
        RETURN;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        IF ERROR_NUMBER() = 1205 -- deadlock victim
        BEGIN
            SET @Attempts += 1;
            WAITFOR DELAY '00:00:00.1'; -- back off
        END
    END CATCH
END
```

```

        CONTINUE;
    END
    THROW; -- re-raise non-deadlock errors
END CATCH
END

```

30. Execution Plans

An execution plan is the strategy SQL Server uses to execute a query: which indexes to seek, which join algorithms to use, in what order, and how to aggregate. Reading execution plans is the single most important skill for query tuning.

30.1 Estimated vs Actual Plans

Plan Type	How Produced	When to Use
Estimated	Produced without executing	Use for query design review
Actual	Includes runtime stats (rows, rebinds, rewinds)	Best for tuning

30.2 Generating Plans

```

-- In SSMS: Ctrl+M toggles "Include Actual Execution Plan"
-- Then run the query; plan appears in a separate tab.

-- In T-SQL: get plan as XML
SET STATISTICS XML ON;
SELECT * FROM dbo.Orders WHERE CustomerID = 42;
SET STATISTICS XML OFF;

-- Get plan from cache (find what the app actually ran)
SELECT SUBSTRING(st.text, (qs.statement_start_offset/2)+1,
    (CASE qs.statement_end_offset
        WHEN -1 THEN DATALENGTH(st.text)
        ELSE qs.statement_end_offset
    END - qs.statement_start_offset)/2 + 1) AS SQLText,
    qp.query_plan,
    qs.execution_count,
    qs.total_logical_reads,
    qs.total_elapsed_time / 1000.0 AS TotalSeconds
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
ORDER BY qs.total_logical_reads DESC;

```

30.3 Common Operators

Operator	What It Does	Notes
Index Seek	B-tree traversal using key	Best — fast
Index Scan	Read entire index leaf level	OK for small indexes; bad for large

Operator	What It Does	Notes
Table Scan	Read entire heap (no clustered index)	Usually bad
Key Lookup	Fetch non-key columns from clustered index	Often a perf killer
Nested Loops Join	For each outer row, seek inner	Best for small outer
Hash Match Join	Build hash table on smaller input, probe with larger	Best for large inputs
Merge Join	Both inputs sorted, merge them	Best when inputs are sorted
Sort	In-memory or tempdb spill	Avoid by adding index
Filter	Apply predicate	Normal
Compute Scalar	Calculate expression	Normal
Parallelism (Gather Streams)	Merge parallel threads	CPU heavy; check if needed

30.4 Reading a Plan

- **Read right-to-left, top-to-bottom** — The rightmost operators are the data sources; leftmost is the result.
- **Thicker arrows = more rows** — A thick arrow at the source that thins at the output indicates a selective filter — good. A thick arrow throughout means lots of data is flowing.
- **Cost percentages** — Operator cost is relative within the plan. High-cost operators are the tuning targets.
- **Actual vs Estimated rows** — Big difference = stale statistics. Update stats.
- **Warnings** — Yellow triangle = spill to tempdb, missing index, or other issue.

30.5 Missing Index Hints

```
-- SQL Server suggests missing indexes in the plan XML and DMV:
SELECT migs.avg_total_user_cost * migs.avg_user_impact * (migs.user_seeks +
migs.user_scans) AS ImprovementMeasure,
       mid.statement AS TableName,
       mid.equality_columns,
       mid.inequality_columns,
       mid.included_columns
FROM sys.dm_db_missing_index_groups mig
JOIN sys.dm_db_missing_index_group_stats migs ON migs.group_handle =
mig.index_group_handle
JOIN sys.dm_db_missing_index_details mid ON mid.index_handle = mig.index_handle
ORDER BY ImprovementMeasure DESC;

-- ▲ Do NOT blindly add every suggested index. Review, prioritise, test.
```


31. Query Optimization

Query optimisation is the iterative process of finding and fixing performance bottlenecks: scans that should be seeks, missing indexes, badly-written predicates, suboptimal joins. The optimiser does most of the work, but it cannot fix non-sargable queries or missing indexes.

31.1 Sargability (Search Argument-able)

A predicate is "sargable" if the optimiser can use an index to satisfy it. Wrapping a column in a function breaks sargability — the optimiser must scan.

Bad (Non-sargable)	Issue	Good (Sargable)
WHERE YEAR(d) = 2026	Non-sargable	WHERE d >= '2026-01-01' AND d < '2027-01-01'
WHERE LEFT(name, 3) = 'abc'	Non-sargable	WHERE name LIKE 'abc%'
WHERE ISNULL(col, 0) = 5	Non-sargable	WHERE col = 5 OR (col IS NULL AND 5 = 0)
WHERE col + 1 = 6	Non-sargable	WHERE col = 5
WHERE col LIKE '%abc%'	Non-sargable (leading wildcard)	Use Full-Text Search
WHERE col = @p	Sargable	—
WHERE col BETWEEN 1 AND 10	Sargable	—
WHERE col IN (1,2,3)	Sargable	—

31.2 Common Optimisation Techniques

- **Add missing indexes** — Use the missing-index DMVs and execution plan warnings. One good index can be a 1000× speedup.
- **Make predicates sargable** — Rewrite WHERE clauses to avoid functions on columns.
- **Cover with INCLUDE** — Add INCLUDE columns to indexes used by hot queries to avoid key lookups.
- **Replace cursors with set-based operations** — UPDATE...FROM, MERGE, window functions instead of row-by-row cursors.
- **Use temp tables for intermediate results** — Helps the optimiser with complex queries; provide statistics.
- **Avoid SELECT *** — Force the optimiser to read all columns; if you only need 3, name them — may enable a covering index.
- **Batch large operations** — Process 5,000 rows at a time, not 5M, to limit transaction size, log growth, and lock escalation.
- **Update statistics** — Out-of-date stats cause bad cardinality estimates. Auto-update is async by default; force with UPDATE STATISTICS table.

31.3 Temp Table vs CTE for Performance

```
-- CTE: inlined, no materialisation, no stats – optimiser may run once per
-- reference. Good for readability and recursive queries.
WITH Heavy AS (SELECT ... FROM BigTable WHERE ...)
SELECT * FROM Heavy h1 JOIN Heavy h2 ON ...; -- Δ Heavy runs twice!

-- Temp table: materialised in tempdb with statistics – runs once.
SELECT * INTO #Heavy FROM BigTable WHERE ...;
SELECT * FROM #Heavy h1 JOIN #Heavy h2 ON ...; -- #Heavy read twice, OK.

-- Rule of thumb: CTE for readability, temp table when referenced 2+ times or
-- when the optimiser produces a bad plan from a complex CTE.
```

31.4 Window Functions vs Self-Joins

```
-- Slow: correlated subquery (per-row evaluation)
SELECT o.OrderID, o.Total,
       (SELECT MAX(Total) FROM dbo.Orders WHERE CustomerID = o.CustomerID) AS CustMax
FROM dbo.Orders o;

-- Fast: window function (single pass)
SELECT OrderID, Total,
       MAX(Total) OVER (PARTITION BY CustomerID) AS CustMax
FROM dbo.Orders;
```

32. Normalization

Normalisation is the process of organising a schema to reduce data redundancy and improve integrity. Each "normal form" is a rule that eliminates a specific kind of anomaly. Most OLTP databases target 3NF or BCNF; data warehouses often denormalise for read performance.

32.1 Normal Forms

Form	Rule	Effect
1NF	Atomic values; no repeating groups	Each cell holds one value; primary key defined
2NF	1NF + no partial dependency on composite key	All non-key attributes depend on the WHOLE key
3NF	2NF + no transitive dependency	Non-key attributes depend only on the key, not on other non-key attributes
BCNF	Stricter 3NF	Every determinant is a candidate key
4NF	No multi-valued dependencies	Rare in practice

32.2 1NF Example

```
-- Bad (not 1NF): comma-separated list in one column
CREATE TABLE dbo.Orders (
    OrderID INT, Products VARCHAR(500) -- "1,2,5,7"
);
```

```
-- Good (1NF): each product in its own row
CREATE TABLE dbo.Orders (
    OrderID INT, ProductID INT, Quantity INT,
    PRIMARY KEY (OrderID, ProductID)
);
```

32.3 2NF Example

```
-- Bad (partial dependency): OrderID+ProductID is PK, but ProductName depends
-- only on ProductID, not on the whole composite key.
CREATE TABLE dbo.OrderItems (
    OrderID INT,
    ProductID INT,
    Quantity INT,
    ProductName NVARCHAR(100), -- depends on ProductID only
    PRIMARY KEY (OrderID, ProductID)
);

-- Good (2NF): split into two tables
CREATE TABLE dbo.OrderItems (
    OrderID INT, ProductID INT, Quantity INT,
    PRIMARY KEY (OrderID, ProductID),
    FOREIGN KEY (ProductID) REFERENCES dbo.Products(ProductID)
);
CREATE TABLE dbo.Products (
    ProductID INT PRIMARY KEY,
    Name NVARCHAR(100)
);
```

32.4 3NF Example

```
-- Bad (transitive dependency): CustomerID -> ZipCode -> City. City depends on
-- CustomerID transitively through ZipCode.
CREATE TABLE dbo.Customers (
    CustomerID INT PRIMARY KEY,
    Name NVARCHAR(100),
    ZipCode VARCHAR(10),
    City NVARCHAR(50) -- depends on ZipCode, not directly on CustomerID
);

-- Good (3NF): separate ZipCode -> City lookup
CREATE TABLE dbo.Customers (
    CustomerID INT PRIMARY KEY,
    Name NVARCHAR(100),
    ZipCode VARCHAR(10)
);
CREATE TABLE dbo.ZipCodes (
    ZipCode VARCHAR(10) PRIMARY KEY,
    City NVARCHAR(50),
    State CHAR(2)
);
```

33. Denormalization

Denormalisation is the deliberate introduction of redundancy to improve read performance at the cost of write complexity. It is the opposite of normalisation but not its enemy — they serve different goals (integrity vs read speed). Data warehouses, reporting systems, and read-heavy OLTP tables all use denormalisation.

33.1 When to Denormalise

- **Read-heavy workloads** — When SELECT frequency dwarfs INSERT/UPDATE/DELETE.
- **Expensive joins** — Queries that join 5+ tables on every request.
- **Aggregated reporting** — Pre-computed summaries in a summary table.
- **Geographic distribution** — Star schema in dimensional modeling (fact + dimensions).
- **Historical snapshots** — Storing the price-at-time-of-order in the order line, not just a product FK — needed because product price changes.

33.2 Techniques

Technique	What It Does	Maintenance
Derived column	Store computed value (e.g., OrderTotal)	Updated via trigger or app
Summary table	Pre-aggregated rows (daily sales by region)	Refreshed by job
Materialised view	Indexed view (SQL Server)	Auto-maintained by engine
Star schema	Fact + dimension tables in DW	Joins are simple, fast
Duplicate column	Store CustomerName in Orders as well as Customers	Update via trigger
Column store index	Columnar storage for analytics	5-10× compression + speed

33.3 Denormalisation Trade-offs

- **Faster reads** — Fewer joins, pre-computed values.
- **Slower writes** — Every update may touch multiple places.
- **Anomaly risk** — If a denormalised value is not updated, data is inconsistent. Use triggers or scheduled jobs to keep in sync.
- **Storage cost** — Redundant data takes space (usually acceptable).
- **Complexity** — More moving parts; harder to reason about correctness.

33.4 Example: Snapshot Column

```
-- Snapshot: store the price at time of order (historical accuracy)
CREATE TABLE dbo.OrderItems (
    OrderID INT NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
```

```

UnitPrice DECIMAL(10,2) NOT NULL, -- snapshot at order time
ProductName NVARCHAR(200) NOT NULL, -- snapshot at order time
LineTotal AS (Quantity * UnitPrice) PERSISTED, -- computed column
PRIMARY KEY (OrderID, ProductID),
FOREIGN KEY (ProductID) REFERENCES dbo.Products(ProductID),
FOREIGN KEY (OrderID) REFERENCES dbo.Orders(OrderID)
);

-- Even if Product.Price or Product.Name changes later, the order line shows
-- what was actually sold.

```

34. Keys in SQL Server

Keys are columns (or column sets) that uniquely identify rows. SQL Server supports several key types — primary, alternate (unique), foreign, candidate, composite, surrogate, and natural. Choosing the right key type affects performance, integrity, and schema evolution.

34.1 Key Types

Key Type	Definition	Purpose
Primary Key	Unique + NOT NULL; one per table; clustered by default	Row identifier
Alternate Key (UNIQUE)	Unique; allows one NULL	Secondary unique identifier
Foreign Key	References PK/UNIQUE of another table	Relationship
Composite Key	Multiple columns form the key	Junction tables, natural keys
Surrogate Key	System-generated (IDENTITY, GUID)	Internal row identifier
Natural Key	Business-meaningful (SSN, SKU, email)	Business identifier
Candidate Key	Any column that could be a PK	Choose one as PK

34.2 Surrogate vs Natural Keys

Type	Source	Trade-off
Surrogate	System-generated (INT IDENTITY, GUID)	No business meaning; never changes
Natural	Business-meaningful (email, SSN, SKU)	Stable, but may change

- **Surrogate advantages** — Never changes (no cascade updates); narrow (INT = 4 bytes); clustered index is sequential (no page splits).
- **Surrogate disadvantages** — No business meaning (need a UNIQUE constraint on the natural key); extra column.
- **Natural advantages** — No extra column; meaningful in queries.

- **Natural disadvantages** — May change (forces cascade updates); often wide (email = 100+ bytes = larger indexes); may have legal/privacy implications (SSN).
- **Best practice** — Use a surrogate INT IDENTITY as PK; add UNIQUE constraints for natural keys (SKU, email).

34.3 GUIDs as Primary Keys

- **Random GUIDs cause page splits** — NEWID() produces random values; clustered index page splits on every insert. Use NEWSEQUENTIALID() instead for clustered GUIDs.
- **Wide key** — 16 bytes vs 4 bytes for INT — every nonclustered index stores the clustered key, so all indexes grow 12 bytes per row.
- **When to use** — Distributed systems where IDs are generated on multiple nodes (no central counter), or when you must know the ID before INSERT.
- **Best practice** — Use INT or BIGINT IDENTITY for clustered PK; if you must use GUID, use NEWSEQUENTIALID() or generate them in a sortable pattern (e.g., comb GUIDs).

34.4 Composite Keys

```
-- Junction table for many-to-many: Student ↔ Course
CREATE TABLE dbo.Enrollments (
    StudentID INT NOT NULL,
    CourseID INT NOT NULL,
    EnrolledAt DATE NOT NULL DEFAULT SYSUTCDATETIME(),
    Grade DECIMAL(3,2) NULL,
    CONSTRAINT PK_Enrollments PRIMARY KEY CLUSTERED (StudentID, CourseID),
    CONSTRAINT FK_Enroll_Student FOREIGN KEY (StudentID) REFERENCES
dbo.Students(StudentID),
    CONSTRAINT FK_Enroll_Course FOREIGN KEY (CourseID) REFERENCES
dbo.Courses(CourseID)
);

-- The first column of the composite PK supports seeks on StudentID.
-- To seek on CourseID, add: CREATE INDEX IX_Enroll_Course ON
dbo.Enrollments(CourseID);
```

35. Identity Column

An IDENTITY column is an auto-incrementing integer column. SQL Server generates the next value on each INSERT — the application does not supply it. IDENTITY is the most common way to create surrogate primary keys.

35.1 Basic Syntax

```
CREATE TABLE dbo.Orders (
    OrderID INT IDENTITY(1,1) PRIMARY KEY, -- start at 1, increment by 1
    CustomerID INT NOT NULL,
    OrderDate DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    Total DECIMAL(10,2) NOT NULL
);

-- IDENTITY (seed, increment)
-- IDENTITY(1,1) → 1, 2, 3, 4, ...
```

```
-- IDENTITY(1000,5) → 1000, 1005, 1010, ...
-- IDENTITY(-100,1) → -100, -99, -98, ...
```

35.2 Retrieving Generated Values

```
-- After a single-row INSERT
INSERT INTO dbo.Orders (CustomerID, Total) VALUES (42, 100);
SELECT SCOPE_IDENTITY(); -- safest for single-statement
SELECT @@IDENTITY; -- Δ includes triggers; avoid
SELECT IDENT_CURRENT('dbo.Orders'); -- any session

-- For multi-row INSERT, use OUTPUT clause
INSERT INTO dbo.Orders (CustomerID, Total)
OUTPUT inserted.OrderID, inserted.CustomerID
VALUES (42, 100), (43, 200), (44, 300);
```

35.3 IDENTITY Insert and Reseed

```
-- Allow explicit IDENTITY values (e.g., data import with specific IDs)
SET IDENTITY_INSERT dbo.Orders ON;
INSERT INTO dbo.Orders (OrderID, CustomerID, Total) VALUES (10000, 42, 100);
SET IDENTITY_INSERT dbo.Orders OFF;

-- Reset the IDENTITY seed (e.g., after deleting all rows)
DBCC CHECKIDENT ('dbo.Orders', RESEED, 0); -- next INSERT = 1
-- Δ If you RESEED to a value below existing rows, you will get duplicate keys.
```

35.4 IDENTITY Limitations

- **Not transactional** — A rolled-back INSERT does NOT return the IDENTITY value — gaps appear. This is by design; never assume contiguous IDs.
- **Replication issues** — In merge replication, two nodes may generate the same IDENTITY. Use ranges (assign each node a seed range) or switch to GUIDs.
- **No native cross-table** — Two tables cannot share an IDENTITY sequence. Use a SEQUENCE for that.
- **Cannot be altered** — You cannot add IDENTITY to an existing column. Must create a new table, copy data, swap names.
- **Data Type ceiling** — INT caps at ~2.1B; BIGINT at $\sim 9.2 \times 10^{18}$. Plan for BIGINT if the table may grow beyond INT range.

36. Sequences

A SEQUENCE is a user-defined schema-bound object that generates a sequence of numeric values according to a specification. Unlike IDENTITY, a sequence is independent of any table — you can use the same sequence across multiple tables or in application logic.

36.1 Creating and Using

```
CREATE SEQUENCE dbo.OrderSeq
AS INT
START WITH 1
INCREMENT BY 1
```

```

    MINVALUE 1
    MAXVALUE 999999999
    NO CYCLE
    NO CACHE; -- or CACHE 50 for performance (gaps possible on crash)
GO

-- Get the next value (no INSERT needed)
SELECT NEXT VALUE FOR dbo.OrderSeq AS NewID;

-- Use in INSERT
INSERT INTO dbo.Orders (OrderID, CustomerID, Total)
VALUES (NEXT VALUE FOR dbo.OrderSeq, 42, 100);

-- Use as a default constraint
CREATE TABLE dbo.Orders2 (
    OrderID INT NOT NULL DEFAULT (NEXT VALUE FOR dbo.OrderSeq),
    CustomerID INT,
    Total DECIMAL(10,2)
);

```

36.2 Sequence vs IDENTITY

Property	SEQUENCE	IDENTITY
Scope	Standalone object	Bound to one column in one table
Get value without INSERT	Yes (NEXT VALUE FOR)	No
Share across tables	Yes	No
Cycling	Yes (CYCLE)	No
Cache	Yes (CACHE N)	No (always logs)
Reset / reseed	ALTER SEQUENCE RESTART	DBCC CHECKIDENT RESEED
ANSI standard	Yes	No (SQL Server-specific)

36.3 Use Cases

- **Cross-table surrogate keys** — When two tables must not share ID values (e.g., Orders and Invoices with a single "Document" identity).
- **Pre-INSERT value retrieval** — Generate an ID, use it in business logic, then INSERT with that ID.
- **Cycling counters** — Reset yearly (e.g., invoice numbers per year).
- **Range allocation** — Use `sp_sequence_get_range` to grab a block of N values for bulk insert without round-trips.
- **Multi-tenant sharding** — Assign each tenant a non-overlapping range from a master sequence.

37. Pagination

Pagination returns a subset of results — typically for UI display. SQL Server offers three approaches: OFFSET/FETCH (modern, SQL Server 2012+), ROW_NUMBER (older but flexible), and TOP (top-N only).

37.1 OFFSET / FETCH (Recommended)

```
-- Page 3 of 20 rows per page
SELECT OrderID, CustomerID, OrderDate, Total
FROM dbo.Orders
ORDER BY OrderDate DESC, OrderID -- deterministic order is mandatory
OFFSET 40 ROWS -- skip 40 rows
FETCH NEXT 20 ROWS ONLY; -- take 20

-- Parameters (variable or parameterised)
DECLARE @Page INT = 3, @Size INT = 20;
SELECT * FROM dbo.Orders
ORDER BY OrderDate DESC
OFFSET (@Page - 1) * @Size ROWS
FETCH NEXT @Size ROWS ONLY;
```

37.2 ROW_NUMBER (Older Pattern)

```
-- Useful when you need the row number in the result too, or complex partitioning
WITH Paged AS (
    SELECT OrderID, CustomerID, OrderDate, Total,
           ROW_NUMBER() OVER (ORDER BY OrderDate DESC, OrderID) AS RowNum
    FROM dbo.Orders
)
SELECT * FROM Paged
WHERE RowNum BETWEEN 41 AND 60 -- rows 41-60
ORDER BY RowNum;
```

37.3 Keyset Pagination (High-Performance)

```
-- For deep pagination, OFFSET scans-and-skips all prior rows – O(N). Keyset
-- pagination uses the last seen value as a bookmark – O(log N).
-- Client passes the last OrderDate+OrderID from the previous page.

DECLARE @LastDate DATETIME2, @LastID INT;
-- ... set @LastDate, @LastID from previous page ...

SELECT TOP (20) OrderID, CustomerID, OrderDate, Total
FROM dbo.Orders
WHERE (OrderDate < @LastDate)
    OR (OrderDate = @LastDate AND OrderID > @LastID)
ORDER BY OrderDate DESC, OrderID ASC;

-- △ Requires a supporting index: (OrderDate DESC, OrderID ASC) INCLUDE (Total).
```

37.4 Returning Total Count

```
-- Compute total rows for "Page 3 of 50" UI without a second round-trip:
SELECT OrderID, CustomerID, OrderDate, Total,
       COUNT(*) OVER () AS TotalCount -- total rows matching WHERE
FROM dbo.Orders
WHERE CustomerID = 42
ORDER BY OrderDate DESC
```

```
OFFSET 40 ROWS FETCH NEXT 20 ROWS ONLY;
```

```
-- Δ COUNT(*) OVER () on a large table can be expensive. Consider caching the
-- total or using an approximate count for very large tables.
```

38. Dynamic SQL

Dynamic SQL is T-SQL constructed as a string at runtime and then executed. It is essential when the table name, column list, or WHERE clause varies per call (e.g., a search proc with optional filters). It also brings SQL injection risk — always parameterise.

38.1 Basic Construction

```
DECLARE @SQL NVARCHAR(MAX);
SET @SQL = N'SELECT * FROM dbo.Customers WHERE Country = ''US''';
EXEC sp_executesql @SQL;

-- Better: parameterised
DECLARE @Country NVARCHAR(50) = N'US';
SET @SQL = N'SELECT * FROM dbo.Customers WHERE Country = @p';
EXEC sp_executesql @SQL, N'@p NVARCHAR(50)', @p = @Country;
```

38.2 sp_executesql vs EXEC

Method	Parameterised	Plan Reuse	Type Safety	Use
sp_executesql	Yes (parameters)	Strong	Strongly typed	Recommended
EXEC (...)	No (string concat)	Weak (injection)	String only	Avoid for user input

38.3 Dynamic Search Proc

```
CREATE OR ALTER PROCEDURE dbo.SearchOrders
    @CustomerID INT = NULL,
    @StartDate DATE = NULL,
    @EndDate DATE = NULL,
    @MinTotal DECIMAL(10,2) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @SQL NVARCHAR(MAX) = N'
        SELECT OrderID, CustomerID, OrderDate, Total
        FROM dbo.Orders
        WHERE 1 = 1';

    IF @CustomerID IS NOT NULL
        SET @SQL += N' AND CustomerID = @CustomerID';
    IF @StartDate IS NOT NULL
        SET @SQL += N' AND OrderDate >= @StartDate';
    IF @EndDate IS NOT NULL
        SET @SQL += N' AND OrderDate < DATEADD(day, 1, @EndDate)';
    IF @MinTotal IS NOT NULL
        SET @SQL += N' AND Total >= @MinTotal';
```

```

SET @SQL += N' ORDER BY OrderDate DESC';

EXEC sp_executesql @SQL,
    N'@CustomerID INT, @StartDate DATE, @EndDate DATE, @MinTotal DECIMAL(10,2)',
    @CustomerID, @StartDate, @EndDate, @MinTotal;

END;

```

38.4 SQL Injection Prevention

- **Never concatenate user input** — Even quoting is unsafe — QUOTENAME helps for identifiers, but parameters are always safer.
- **Use sp_executesql with parameters** — Treats user input as values, not SQL syntax. Injection impossible.
- **QUOTENAME for identifiers** — When you must use a dynamic table/column name: SET @SQL = N'SELECT * FROM ' + QUOTENAME(@TableName). QUOTENAME wraps in [...] and escapes embedded brackets.
- **Permissions** — Grant EXECUTE on stored procs; deny direct table access. Even if injection succeeds, the attacker cannot do anything the proc cannot do.

39. XML and JSON

SQL Server has native support for XML (since 2005) and JSON (since 2016). XML has a dedicated data type with XQuery support; JSON has no native type — store as NVARCHAR(MAX) and use JSON_VALUE / OPENJSON / JSON_MODIFY functions.

39.1 XML Data Type

```

CREATE TABLE dbo.Books (
    BookID INT PRIMARY KEY,
    Title NVARCHAR(200),
    Metadata XML -- typed column needs XML schema collection
);

INSERT INTO dbo.Books VALUES
(1, N'SQL Basics', N'Alice2024');

-- Query XML with XQuery
SELECT Title,
    Metadata.value('(/author)[1]', 'NVARCHAR(100)') AS Author,
    Metadata.value('(/year)[1]', 'INT') AS Year,
    Metadata.value('(/book/@isbn)[1]', 'VARCHAR(20)') AS ISBN
FROM dbo.Books;

-- XML indexes (primary + secondary) for fast queries on large XML columns

```

39.2 JSON in SQL Server

```

-- JSON is stored as NVARCHAR(MAX); no native JSON type
CREATE TABLE dbo.Events (
    EventID INT IDENTITY PRIMARY KEY,
    EventTime DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),

```

```

    Payload NVARCHAR(MAX) NOT NULL
    CHECK (ISJSON(Payload) = 1) -- validate on insert
);

INSERT INTO dbo.Events (Payload) VALUES
(N'{"event":"signup","user":"alice","props":{"plan":"pro","country":"US"}}');

-- Extract a scalar value
SELECT JSON_VALUE(Payload, '$.event') AS EventType,
       JSON_VALUE(Payload, '$.user') AS UserName,
       JSON_VALUE(Payload, '$.props.plan') AS Plan
FROM dbo.Events;

-- OPENJSON: expand JSON array/object into rows
DECLARE @json NVARCHAR(MAX) = N'[{ "id":1,"name":"A"},{ "id":2,"name":"B"}]';
SELECT * FROM OPENJSON(@json)
WITH (id INT, name NVARCHAR(100));

-- Modify JSON
UPDATE dbo.Events
SET Payload = JSON_MODIFY(Payload, '$.props.plan', 'enterprise')
WHERE EventID = 1;

-- Format query results as JSON
SELECT OrderID, CustomerID, OrderDate, Total
FROM dbo.Orders
WHERE CustomerID = 42
FOR JSON PATH; -- produces a JSON array

```

39.3 XML vs JSON

Aspect	XML	JSON
Native type	Yes (XML)	No (NVARCHAR(MAX))
Validation	XML schema collection	ISJSON() CHECK constraint
Indexing	Primary + secondary XML indexes	Computed columns + regular indexes
Query language	XQuery / .value() / .query()	JSON_VALUE / JSON_QUERY / OPENJSON
Storage	Compact (binary XML)	Plain text
Modern preference	Avoid for new work	Default for new apps

40. Error Handling

T-SQL error handling uses TRY...CATCH (since SQL Server 2005). Inside a CATCH block, you can inspect the error with functions like ERROR_NUMBER, ERROR_MESSAGE, ERROR_LINE, and re-raise with THROW.

40.1 TRY...CATCH Structure

```

BEGIN TRY
    -- Code that may fail
    INSERT INTO dbo.Orders (CustomerID, Total) VALUES (42, 100);
END TRY
BEGIN CATCH
    -- Inspect the error
    DECLARE @ErrNum INT = ERROR_NUMBER();
    DECLARE @ErrMsg NVARCHAR(4000) = ERROR_MESSAGE();
    DECLARE @ErrSev INT = ERROR_SEVERITY();
    DECLARE @ErrState INT = ERROR_STATE();
    DECLARE @ErrProc NVARCHAR(200) = ERROR_PROCEDURE();
    DECLARE @ErrLine INT = ERROR_LINE();

    -- Log to an error table
    INSERT INTO dbo.ErrorLog (ErrorNumber, ErrorMessage, ProcedureName, ErrorLine,
        LoggedAt)
        VALUES (@ErrNum, @ErrMsg, @ErrProc, @ErrLine, SYSUTCDATETIME());

    -- Re-raise to caller
    THROW;
END CATCH;

```

40.2 ERROR_* Functions

Function	Returns	Notes
ERROR_NUMBER()	INT	Error number (sys.messages.message_id)
ERROR_SEVERITY()	INT	0-25 (≥20 = fatal connection drop)
ERROR_STATE()	INT	Specific to error; helps Microsoft support
ERROR_PROCEDURE()	NVARCHAR(128)	Proc or trigger where error occurred
ERROR_LINE()	INT	Line number (1-based) within the proc
ERROR_MESSAGE()	NVARCHAR(4000)	Human-readable message

40.3 THROW vs RAISERROR

```

-- THROW (SQL Server 2012+) – preferred in CATCH blocks
-- Re-raises the caught error preserving severity, state, line, and proc:
BEGIN CATCH
    THROW; -- no args = re-throw current
END CATCH;

-- THROW with custom values (always severity 16):
THROW 50000, N'Custom error: customer not found', 1;

-- RAISERROR – older, more flexible but more verbose
RAISERROR(N'Customer %d not found', 16, 1, @CustomerID);

-- WITH NOWAIT sends immediately to client (no output buffering):
RAISERROR(N'Step 1 done', 0, 1) WITH NOWAIT;

-- THROW advantages:
-- - Preserves original line number

```

```
-- - Always severity 16 (consistent)
-- - Simpler syntax
-- RAISERROR advantages:
-- - Severity < 11 for informational messages
-- - WITH NOWAIT for progress reporting
-- - WITH LOG for server-wide alerts
```

40.4 Custom Error Messages

```
-- Define a custom message once (in master)
EXEC sp_addmessage
    @msgnum = 50001,
    @severity = 16,
    @msgtext = N'Customer %d does not exist';

-- Use it
RAISERROR(50001, 16, 1, @CustomerID);
-- or THROW 50001, ... (but THROW does not support printf-style substitution)
```

40.5 XACT_ABORT and Transaction State

```
-- SET XACT_ABORT ON ensures any error rolls back the entire transaction,
-- including errors that bypass TRY/CATCH (e.g., conversion errors).
CREATE OR ALTER PROCEDURE dbo.SafeUpdate
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON; -- ▲ always in production procs

    BEGIN TRY
        BEGIN TRANSACTION;
        -- ... work ...
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        -- XACT_STATE() = -1 → uncommittable, must rollback
        -- XACT_STATE() = 1 → committable, can choose
        IF XACT_STATE() = -1
            ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

41. Backup and Restore

Backups are the last line of defence against data loss. SQL Server offers full, differential, and transaction log backups. Combined with the right recovery model, you can restore to any point in time down to the second.

41.1 Backup Types

Type	What It Captures	Purpose	Typical Frequency
Full	Entire database	Base for diffs and logs	Weekly/daily
Differential	Changes since last full	Faster than full, smaller	Daily

Type	What It Captures	Purpose	Typical Frequency
Transaction Log	All log records since last log backup	Allows PITR	Every 5-15 min
Copy-only Full	Full backup that does NOT reset diff base	Out-of-band copies	Ad hoc
File / Filegroup	Specific files only	VLDB partial backup	Partitioned DBs

41.2 Recovery Models

Model	Log Behaviour	Point-in-Time Restore	Log Backups	Use Case
SIMPLE	Truncate log on checkpoint	No	No	Dev/test, non-critical
FULL	Log preserved until log backup	Yes (down to second)	Yes	Production OLTP
BULK_LOGGED	Minimal logging for bulk ops	Limited (PITR for non-bulk)	Yes	During bulk loads

41.3 Backup Commands

```
-- Full backup (compress + checksum)
BACKUP DATABASE SalesDB
TO DISK = N'D:\Backup\SalesDB_FULL_20260703.bak'
WITH FORMAT, INIT, COMPRESSION, CHECKSUM,
     NAME = N'SalesDB-Full-20260703';

-- Differential backup
BACKUP DATABASE SalesDB
TO DISK = N'D:\Backup\SalesDB_DIFF_20260703.bak'
WITH INIT, COMPRESSION, CHECKSUM,
     DIFFERENTIAL;

-- Transaction log backup (requires FULL or BULK_LOGGED recovery model)
BACKUP LOG SalesDB
TO DISK = N'D:\Backup\SalesDB_LOG_20260703_1415.trn'
WITH INIT, COMPRESSION, CHECKSUM;
```

41.4 Restore Sequence

```
-- Restore a FULL backup (NORECOVERY = more backups to come)
RESTORE DATABASE SalesDB
FROM DISK = N'D:\Backup\SalesDB_FULL_20260701.bak'
WITH NORECOVERY,
     REPLACE,
     STATS = 10;

-- Restore the latest DIFFERENTIAL (still NORECOVERY)
RESTORE DATABASE SalesDB
FROM DISK = N'D:\Backup\SalesDB_DIFF_20260703.bak'
WITH NORECOVERY;
```

```
-- Restore transaction logs up to the desired point (RECOVERY = done)
RESTORE LOG SalesDB
FROM DISK = N'D:\Backup\SalesDB_LOG_20260703_1400.trn'
WITH NORECOVERY;

RESTORE LOG SalesDB
FROM DISK = N'D:\Backup\SalesDB_LOG_20260703_1415.trn'
WITH RECOVERY, -- last log = bring database online
STOPAT = N'2026-07-03T14:12:30'; -- PITR to a specific time
```

41.5 Common Backup Strategy (OLTP)

- **Full backup weekly (Sunday 2am)** — Base for the week.
- **Differential backup daily (2am)** — Captures changes since Sunday — limits log restore count.
- **Transaction log backup every 15 minutes** — Point-in-time recovery down to 15-minute RPO. Log file size stays bounded.
- **Off-site copies** — Copy backups to Azure Blob Storage / S3 via BACKUP TO URL or a backup tool.
- **Restore tests** — Monthly restore to a test server — a backup you have not tested is not a backup.

42. SQL Server Agent

SQL Server Agent is the built-in job scheduler. It runs scheduled T-SQL jobs, SSIS packages, PowerShell scripts, and replication maintenance. The Agent service must be running for jobs to execute.

42.1 Core Concepts

Object	Purpose	Notes
Job	A unit of work containing one or more steps	Created per task
Step	A single operation (T-SQL, SSIS, PowerShell, CmdExec)	Sequential or conditional
Schedule	When the job runs (daily, weekly, on idle, on alert)	Reusable across jobs
Operator	A person/contact for notifications	Email, pager, net send
Alert	Action triggered by an event (error, perf counter)	Run job or notify operator
Proxy	Security context for non-T-SQL steps	Credentials for SSIS/PowerShell

42.2 Creating a Job in T-SQL

```
-- 1. Operator (notification target)
EXEC msdb.dbo.sp_add_operator
    @name = N'DBA Team',
    @email_address = N'dba@company.com';

-- 2. Job
EXEC msdb.dbo.sp_add_job
    @job_name = N'Nightly Index Rebuild';
```



```
-- 3. Step (the actual work)
EXEC msdb.dbo.sp_add_jobstep
    @job_name = N'Nightly Index Rebuild',
    @step_name = N'Rebuild fragmented indexes',
    @subsystem = N'TSQL',
    @command = N'EXEC dbo.usp_RebuildIndexes @MaxFrag = 30;',
    @database_name = N'SalesDB';

-- 4. Schedule
EXEC msdb.dbo.sp_add_jobschedule
    @job_name = N'Nightly Index Rebuild',
    @name = N'Nightly 2am',
    @freq_type = 4, -- daily
    @freq_interval = 1, -- every day
    @active_start_time = 020000; -- 02:00:00

-- 5. Notification on failure
EXEC msdb.dbo.sp_add_jobserver @job_name = N'Nightly Index Rebuild';
EXEC msdb.dbo.sp_update_job
    @job_name = N'Nightly Index Rebuild',
    @notify_level_email = 2, -- on failure
    @notify_email_operator_name = N'DBA Team';
```

42.3 Job History and Troubleshooting

```
-- View job history (last 10 days)
SELECT j.name AS JobName,
       h.run_date, h.run_time, h.run_duration,
       CASE h.run_status
           WHEN 0 THEN 'Failed'
           WHEN 1 THEN 'Succeeded'
           WHEN 2 THEN 'Retry'
           WHEN 3 THEN 'Canceled'
           WHEN 4 THEN 'In Progress'
       END AS Status,
       h.message
FROM msdb.dbo.sysjobs j
JOIN msdb.dbo.sysjobhistory h ON h.job_id = j.job_id
WHERE h.step_id = 0 -- job outcome row
      AND h.run_date > CAST(REPLACE(CONVERT(CHAR(8), DATEADD(day, -10, GETDATE()), 112),
    ',') AS INT)
ORDER BY h.run_date DESC, h.run_time DESC;
```

42.4 Common Job Categories

- **Maintenance** — Index rebuild/reorganise, update statistics, integrity checks (DBCC CHECKDB). Use Ola Hallengren's free scripts — industry standard.
- **Backups** — Full/diff/log backups. Ola Hallengren or native maintenance plans.
- **Cleanup** — Purge old data, archive, history tables.
- **ETL** — SSIS package execution, data loads.
- **Monitoring** — Capture perf counters, populate monitoring tables, alert on thresholds.
- **Agent-only (no SQL)** — PowerShell script to copy files, call webhooks, restart services.

43. Security

SQL Server security spans authentication (who can connect), authorisation (what they can do), encryption (data at rest and in transit), auditing (what they did), and compliance (regulatory requirements). A DBA must apply defence in depth — never rely on a single layer.

43.1 Security Layers

Layer	Purpose	Scope
Authentication	Verify identity (Windows / SQL login / Entra ID)	Server scope
Authorisation	Grant/deny permissions on objects	Server + DB scope
Encryption in transit	TLS for client↔server traffic	Force encryption
Encryption at rest	TDE for data files, BitLocker for disk	Transparent to app
Encryption in use	Always Encrypted, Always Encrypted with enclaves	Client drives keys
Row-Level Security	Filter rows by user context	Multi-tenant
Dynamic Data Masking	Mask columns for low-privilege users	PII / GDPR
Auditing	Track who did what when	Compliance

43.2 Authentication Modes

- **Windows Authentication** — Recommended. SQL Server delegates to Active Directory; no passwords in connection strings. Uses Kerberos or NTLM.
- **SQL Authentication** — Username + password stored in SQL Server. Less secure; passwords travel in connection strings. Use only when AD is unavailable.
- **Azure AD (Entra ID)** — Cloud-native. Multi-factor authentication, conditional access. Required for Azure SQL Database.
- **Contained database users** — User lives in the database, not at server level — easier to move databases between servers. Recommended for Azure SQL.

43.3 Encryption Technologies

Technology	What It Protects	Notes
TDE (Transparent Data Encryption)	Encrypts data + log files at rest	Page-level, transparent to app
Always Encrypted	Client encrypts/decrypts; server never sees plaintext	Highest confidentiality
Cell-level encryption	Encrypt specific columns with symmetric/asymmetric keys	App-driven, slower

Technology	What It Protects	Notes
Backup encryption	Encrypt backup files with certificate or asymmetric key	Mandatory for offsite
Connection encryption	TLS 1.2+ for client↔server	Force in Configuration Manager

43.4 Always Encrypted Example

```
-- 1. Create a column master key in the database (points to a key in
-- Azure Key Vault / Windows Certificate Store)
CREATE COLUMN MASTER KEY CMK_Azure
WITH (
    KEY_STORE_PROVIDER_NAME = N'AZURE_KEY_VAULT',
    KEY_PATH = N'https://mykv.vault.azure.net/keys/CMK_Azure/abc123'
);

-- 2. Create a column encryption key (encrypted by the CMK)
CREATE COLUMN ENCRYPTION KEY CEK_Payroll
WITH VALUES (
    COLUMN_MASTER_KEY = CMK_Azure,
    ALGORITHM = 'RSA_OAEP',
    ENCRYPTED_VALUE = 0x0123... -- 256-bit AES key wrapped by CMK
);

-- 3. Create a table with encrypted columns
CREATE TABLE dbo.Employees (
    EmployeeID INT PRIMARY KEY,
    Name NVARCHAR(100),
    SSN NVARCHAR(11) COLLATE Latin1_General_BIN2
        ENCRYPTED WITH (ENCRIPTION_TYPE = DETERMINISTIC,
            ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256',
            COLUMN_ENCRYPTION_KEY = CEK_Payroll),
    Salary DECIMAL(10,2)
        ENCRYPTED WITH (ENCRIPTION_TYPE = RANDOMIZED,
            ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256',
            COLUMN_ENCRYPTION_KEY = CEK_Payroll)
);
```

43.5 Row-Level Security

```
-- Filter predicate: users only see rows where their region matches
CREATE SECURITY POLICY dbo.RegionFilter
ADD FILTER PREDICATE dbo.fn_RegionPredicate(Region) ON dbo.Sales,
ADD BLOCK PREDICATE dbo.fn_RegionBlock(Region) ON dbo.Sales
WITH (STATE = ON);
GO

CREATE FUNCTION dbo.fn_RegionPredicate(@Region AS NVARCHAR(50))
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN SELECT 1 AS fn_result
        WHERE @Region = USER_NAME() -- or look up user-region mapping
        OR USER_NAME() = 'sa'; -- super-user bypass
GO
```

44. Permissions and Roles

SQL Server has a rich permission model: server-level permissions (login, server role), database-level permissions (user, database role), and object-level permissions (SELECT, INSERT, EXECUTE on specific objects). Roles group permissions for easier management.

44.1 Fixed Server Roles

Role	Power	Typical Assignee
sysadmin	Full control of the instance	Avoid for app logins
serveradmin	Server-wide config, SHUTDOWN	DBA junior
securityadmin	Manage logins and permissions	Helpdesk
processadmin	Kill sessions, manage processes	Ops
diskadmin	Manage disk files	Rare
dbcreator	Create/alter/drop databases	Dev env
bulkadmin	Run BULK INSERT	ETL accounts
public	Default, has VIEW ANY DATABASE	Everyone

44.2 Fixed Database Roles

Role	Power	Typical Assignee
db_owner	Full control of the database	DBA / schema owner
db_securityadmin	Manage roles and permissions	Security admin
db_accessadmin	Add/remove database users	Helpdesk
db_backupoperator	Backup the database	Service account
db_ddladmin	Create/modify schema (no data access)	Developer
db_datawriter	INSERT/UPDATE/DELETE on all tables	App write role
db_datareader	SELECT on all tables	App read role / reporting
db_denydatawriter	Deny writes	Compliance
db_denydatareader	Deny reads	Compliance
public	Default, no inherent permissions	Everyone

44.3 GRANT / DENY / REVOKE

```
-- GRANT: allow a permission
GRANT SELECT ON dbo.Customers TO AppReader;
GRANT EXECUTE ON dbo.GetCustomerOrders TO AppReader;
GRANT SELECT ON SCHEMA::Sales TO AppReader; -- schema scope

-- DENY: explicitly forbid (overrides GRANT)
DENY SELECT ON dbo.Salary TO AppReader;

-- REVOKE: remove a GRANT or DENY (back to implicit)
REVOKE SELECT ON dbo.Customers FROM AppReader;

-- WITH GRANT OPTION: allow the user to grant this to others
GRANT SELECT ON dbo.Lookup TO Manager WITH GRANT OPTION;
```

44.4 Application Roles

```
-- Application role: only active when app activates it with password
CREATE APPLICATION ROLE AppReadOnly
    WITH PASSWORD = N'Str0ng!Pass';
GRANT SELECT ON SCHEMA::dbo TO AppReadOnly;

-- App connects as a low-privilege user, then activates the app role:
EXEC sp_setapprole N'AppReadOnly', N'Str0ng!Pass';
-- Now the session has the app role's permissions, not the user's.
```

44.5 Principle of Least Privilege

- **Grant only what is needed** — A read-only reporting login should have SELECT on specific tables or views — not db_datareader on everything.
- **Use stored procs** — Grant EXECUTE on procs, not direct table access. Procs encapsulate logic and prevent SQL injection.
- **Avoid db_owner for apps** — App connection strings should not be db_owner. If the app is compromised, the attacker gets full DB control.
- **Use schemas for grouping** — GRANT SELECT ON SCHEMA::Sales TO SalesTeam. Easy to add new tables in the schema without re-granting.
- **DENY for compliance** — Use DENY to prevent specific users from accessing sensitive data even if they are in a role that grants it.
- **Audit permissions** — Periodically review who has access to what. Use sys.database_permissions and sys.server_permissions.

45. Replication

Replication copies data from a source (Publisher) to one or more destinations (Subscribers) using a Distributor intermediary. Used for scale-out reads, disaster recovery, reporting offload, and integrating geographically distributed databases.

45.1 Replication Types

Type	How	Use Case
Transactional	Log reader ships INSERT/UPDATE/DELETE to subscribers	Near-real-time OLTP scale-out
Snapshot	Full copy of the published data; replaces subscriber entirely	Infrequent, small tables
Merge	Bidirectional; conflict resolution needed	Disconnected clients / mobile
Peer-to-Peer	All nodes publish and subscribe; multi-master	Scale-out writes (Enterprise)

45.2 Replication Architecture

- **Publisher** — Source database with data to replicate. Defines publications (groups of articles = tables/views/procs).
- **Distributor** — Intermediate server that stores replication metadata and queues changes. Can be local (same as Publisher) or remote.
- **Subscriber** — Destination database receiving replicated data.
- **Article** — A single published object (table, view, stored proc). Can be filtered (row/column) for partitioned data.
- **Publication** — A named collection of articles; the unit of subscription.
- **Push subscription** — Distributor agent runs on Distributor, pushes to Subscriber.
- **Pull subscription** — Agent runs on Subscriber, pulls from Distributor. Reduces Distributor load.

45.3 When to Use Replication (vs Alternatives)

- **Always On AG** — For HA/DR with automatic failover and readable secondaries. Replication is for distribution, not HA.
- **Replication** — Scale-out reads (multiple reporting copies), integration across versions/editions, offline sync (merge).
- **Log Shipping** — Simpler DR than AG; warm standby; manual failover.
- **Service Broker / ETL** — For one-way application-level message passing or batch integration.

45.4 Common Issues

- **Latency** — Transactional replication has seconds-to-minutes latency under load. Monitor distribution agent.
- **Schema changes** — ALTER TABLE on Publisher may or may not propagate — depends on article options. Test before applying.
- **Identity range management** — Each subscriber needs a non-overlapping IDENTITY range for merge/P2P. SQL Server auto-manages with ranges.
- **Conflicts (merge)** — When two nodes update the same row, conflict resolver picks a winner. Default: first writer wins. Custom resolvers available.

- **Trigger and constraint considerations** — Subscribers fire triggers; NOT FOR REPLICATION option prevents infinite loops.

46. Always On Availability Groups

Always On Availability Groups (AG) is the premier HA/DR technology for SQL Server. An AG groups 1-100 user databases into a failover unit. The primary replica serves reads/writes; up to 8 secondary replicas can serve reads and act as failover targets. Requires Windows Server Failover Cluster (WSFC) on Windows, or Pacemaker on Linux.

46.1 AG Architecture

- **Availability Group** — Logical container for 1+ databases that fail over together.
- **Primary replica** — The active node serving read/write traffic.
- **Secondary replica** — Up to 8 secondary nodes. Synchronous or asynchronous commit. Can be readable.
- **Availability mode** — SYNCHRONOUS_COMMIT — zero data loss, higher latency.
ASYNCHRONOUS_COMMIT — potential data loss, lower latency (DR scenario).
- **Failover mode** — AUTOMATIC (requires synchronous commit + WSFC health check) or MANUAL.
- **Listener** — A virtual network name (VNN) that clients connect to. Always points to the current primary. Transparent failover for clients.
- **Required: WSFC** — Windows Server Failover Cluster must be set up before AG. On Linux, use Pacemaker.

46.2 AG vs Failover Cluster Instance (FCI)

Aspect	Always On AG	Failover Cluster Instance
Granularity	Per-database group	Whole instance
Shared storage	No (each replica has own copy)	Yes (shared disk/SMB)
Readable secondary	Yes	No (only active node online)
Failover speed	Seconds (DB only)	Minutes (full instance restart)
Multiple secondaries	Up to 8	No (one active node)
Typical use	HA + readable scale-out	Instance-level HA

46.3 Creating an AG (T-SQL)

```
-- 1. Enable Always On on the instance (in SQL Server Configuration Manager)
-- 2. Restore the database on each secondary with NORECOVERY:
RESTORE DATABASE SalesDB FROM DISK = '\\share\SalesDB.bak'
WITH NORECOVERY;
RESTORE LOG SalesDB FROM DISK = '\\share\SalesDB_log.trn'
WITH NORECOVERY;
```

```
-- 3. Create the AG on the primary (after creating the endpoint):
CREATE AVAILABILITY GROUP AG_Sales
WITH (
    AUTOMATED_BACKUP_PREFERENCE = PRIMARY,
    DB_FAILOVER = ON,
    DTC_SUPPORT = NONE
)
FOR DATABASE SalesDB
REPLICA ON
    N'SQLNODE01' WITH (
        ENDPOINT_URL = N'TCP://SQLNODE01.contoso.com:5022',
        AVAILABILITY_MODE = SYNCHRONOUS_COMMIT,
        FAILOVER_MODE = AUTOMATIC,
        SECONDARY_ROLE (ALLOW_CONNECTIONS = READ_ONLY)
    ),
    N'SQLNODE02' WITH (
        ENDPOINT_URL = N'TCP://SQLNODE02.contoso.com:5022',
        AVAILABILITY_MODE = SYNCHRONOUS_COMMIT,
        FAILOVER_MODE = AUTOMATIC,
        SECONDARY_ROLE (ALLOW_CONNECTIONS = READ_ONLY)
    ),
    N'SQLNODE03' WITH (
        ENDPOINT_URL = N'TCP://SQLNODE03.contoso.com:5022',
        AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT, -- DR, no auto failover
        FAILOVER_MODE = MANUAL,
        SECONDARY_ROLE (ALLOW_CONNECTIONS = READ_ONLY)
    );

-- 4. Join secondaries (run on each secondary):
ALTER AVAILABILITY GROUP AG_Sales JOIN;

-- 5. Create the listener (run on primary):
ALTER AVAILABILITY GROUP AG_Sales
    ADD LISTENER N'SQL-AG-LIST' (
        WITH IP ((N'10.0.0.50', N'255.255.255.0')),
        PORT = 1433
    );
```

47. Partitioning

Table partitioning splits a single logical table into multiple physical units (partitions) based on a partitioning column. Each partition can be managed independently — archived, indexed, switched in/out — without affecting the others. Requires Enterprise edition (Developer/Evaluation for testing).

47.1 Why Partition?

- **Manageability** — Archive old data by switching out partitions instead of slow DELETES.
- **Performance** — Partition elimination: queries filtering on the partition key scan only relevant partitions.
- **Parallelism** — Multi-threaded scans across partitions; load small partitions in parallel.
- **Storage tiering** — Old partitions on slow/cheap disks; recent on fast SSDs.
- **Maintenance** — Rebuild one partition at a time instead of the whole table.

47.2 Three Objects

Object	Purpose	Question Answered
Partition function	Defines the boundary values	How to split
Partition scheme	Maps partitions to filegroups	Where each goes
Partitioned table/index	Uses the scheme on a partitioning column	The object

47.3 Creating a Partitioned Table

```
-- 1. Partition function: range-right on OrderDate, yearly boundaries
CREATE PARTITION FUNCTION PF_OrdersByYear(DATETIME2)
AS RANGE RIGHT FOR VALUES
('2024-01-01', '2025-01-01', '2026-01-01', '2027-01-01');
-- 5 partitions: <=2023-12-31, 2024, 2025, 2026, >=2027-01-01

-- 2. Partition scheme: map each partition to a filegroup
CREATE PARTITION SCHEME PS_OrdersByYear
AS PARTITION PF_OrdersByYear
TO (FG_Old, FG_2024, FG_2025, FG_2026, FG_Future);

-- 3. Create table on the scheme
CREATE TABLE dbo.Orders (
    OrderID BIGINT IDENTITY PRIMARY KEY,
    CustomerID INT NOT NULL,
    OrderDate DATETIME2 NOT NULL,
    Total DECIMAL(10,2) NOT NULL
)
ON PS_OrdersByYear(OrderDate); -- partition by OrderDate

-- 4. View partition boundaries and row counts
SELECT $PARTITION.PF_OrdersByYear(OrderDate) AS PartitionNumber,
       COUNT(*) AS Rows
FROM dbo.Orders
GROUP BY $PARTITION.PF_OrdersByYear(OrderDate);
```

47.4 Switching Partitions (Fast Archival)

```
-- Switch out the oldest partition into a staging table (metadata-only op)
CREATE TABLE dbo.Orders_2024_Archive
ON FG_2024 -- must match source partition's filegroup
AS dbo.Orders; -- same schema

ALTER TABLE dbo.Orders
SWITCH PARTITION 2 TO dbo.Orders_2024_Archive; -- instant

-- Now back up the archive table separately, then drop it.
-- The partition in dbo.Orders is now empty; merge to remove the boundary:
ALTER PARTITION FUNCTION PF_OrdersByYear() MERGE RANGE ('2024-01-01');

-- And split to add a new future boundary:
ALTER PARTITION FUNCTION PF_OrdersByYear()
SPLIT RANGE ('2028-01-01');
```

47.5 Partition Elimination

```
-- This query filters on the partitioning column → only scans 2026 partition
SELECT * FROM dbo.Orders
WHERE OrderDate >= '2026-01-01' AND OrderDate < '2027-01-01';

-- This query does NOT filter on OrderDate → scans ALL partitions (slower)
SELECT * FROM dbo.Orders WHERE CustomerID = 42;
```

48. SQL Profiler and Monitoring

Monitoring is the daily work of a DBA. SQL Server ships with many tools: SQL Server Profiler (legacy GUI), Extended Events (modern lightweight tracing), Dynamic Management Views (DMVs) for live state, Query Store for query regression tracking, and the Error Log. Production monitoring should use Extended Events and DMVs — Profiler is deprecated and adds significant overhead.

48.1 Tools Comparison

Tool	Description	Best For
SQL Server Profiler	GUI trace, deprecated	Quick local trace
Extended Events	Lightweight, modern, scalable	Production tracing
DMVs (sys.dm_*)	Live state snapshots, query stats, waits	Ad hoc tuning
Query Store	Per-query plans, stats, regression tracking	8.x onwards, default on
Error Log	Server-level events (startup, errors, backups)	Always on
Performance Monitor (PerfMon)	OS + SQL counters over time	Capacity planning
Activity Monitor (SSMS)	Live sessions, locks, waits	Quick investigation

48.2 Key DMVs

```
-- Top 10 queries by total logical reads (cumulative since last restart)
SELECT TOP 10
    SUBSTRING(st.text, (qs.statement_start_offset/2)+1,
        (CASE qs.statement_end_offset
            WHEN -1 THEN DATALENGTH(st.text)
            ELSE qs.statement_end_offset
        END - qs.statement_start_offset)/2 + 1) AS QueryText,
    qs.execution_count,
    qs.total_logical_reads / qs.execution_count AS AvgLogicalReads,
    qs.total_logical_reads AS TotalLogicalReads,
    qs.total_elapsed_time / 1000.0 AS TotalSeconds,
    qp.query_plan
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
ORDER BY qs.total_logical_reads DESC;
```

48.3 Wait Statistics

```
-- What is the server waiting on? (aggregate since restart)
SELECT TOP 20
    wait_type,
    waiting_tasks_count,
    wait_time_ms / 1000.0 AS WaitSeconds,
    (wait_time_ms - signal_wait_time_ms) / 1000.0 AS ResourceWaitSeconds,
    signal_wait_time_ms / 1000.0 AS SignalWaitSeconds,
    wait_time_ms * 1.0 / SUM(wait_time_ms) OVER () * 100 AS PctOfTotal
FROM sys.dm_os_wait_stats
WHERE wait_type NOT IN (
    'SLEEP_TASK', 'BROKER_TASK_STOP', 'HADR_FILESTREAM_IOMGR_IOCOMPLETION',
    'CHECKPOINT_QUEUE', 'REQUEST_FOR_DEADLOCK_SEARCH', 'XE_DISPATCHER_WAIT',
    'LOGMGR_QUEUE', 'FT_IFTS_SCHEDULER_IDLE_WAIT', 'BROKER_TRANSMITTER',
    'SQLTRACE_BUFFER_FLUSH', 'CLR_AUTO_EVENT', 'BROKER_EVENTHANDLER',
    'LAZYWRITER_SLEEP', 'DIRTY_PAGE_POLL', 'HADR_LOGCAPTURE_WAIT'
)
    AND waiting_tasks_count > 0
ORDER BY wait_time_ms DESC;
```

48.4 Active Sessions and Blocks

```
-- What is running RIGHT NOW?
SELECT er.session_id, er.status, er.wait_type, er.wait_time,
    er.blocking_session_id, er.cpu_time, er.logical_reads,
    SUBSTRING(st.text, (er.statement_start_offset/2)+1,
        (CASE er.statement_end_offset
            WHEN -1 THEN DATALength(st.text)
            ELSE er.statement_end_offset
        END - er.statement_start_offset)/2 + 1) AS QueryText,
    es.login_name, es.host_name, es.program_name
FROM sys.dm_exec_requests er
JOIN sys.dm_exec_sessions es ON es.session_id = er.session_id
CROSS APPLY sys.dm_exec_sql_text(er.sql_handle) st
WHERE er.session_id > 50 -- user sessions only
ORDER BY er.session_id;
```

48.5 Query Store

```
-- Enable (database-level)
ALTER DATABASE SalesDB SET QUERY_STORE = ON;
ALTER DATABASE SalesDB SET QUERY_STORE (
    OPERATION_MODE = READ_WRITE,
    CLEANUP_POLICY = (STALE_QUERY_THRESHOLD_DAYS = 30),
    DATA_FLUSH_INTERVAL_SECONDS = 900,
    MAX_STORAGE_SIZE_MB = 1000,
    QUERY_CAPTURE_MODE = AUTO,
    SIZE_BASED_CLEANUP_MODE = AUTO
);
```

- **Force a good plan** — When a query has plan regression, use SSMS Query Store UI or `sp_query_store_force_plan` to pin the known-good plan.
- **Top resource consumers** — SSMS Reports → Query Store Statistics shows top queries by CPU, reads, duration, execution count, log memory.
- **Plan regression history** — See when a plan changed and which statistics update triggered the change.
- **Default on since SQL Server 2017** — Should be enabled on every production database.

51. Real-Time Optimization Examples

Real production queries and the changes that fixed them. Each example shows the original (slow) query, the diagnosis, the fix, and the measured improvement.

51.1 Non-Sargable Date Filter

```
-- ORIGINAL: 4 minutes, full table scan on 50M-row Orders table
SELECT * FROM dbo.Orders WHERE YEAR(OrderDate) = 2026 AND MONTH(OrderDate) = 7;

-- FIX: range predicate, uses index on OrderDate
SELECT * FROM dbo.Orders
WHERE OrderDate >= '2026-07-01' AND OrderDate < '2026-08-01';

-- Result: 200 ms (1200× faster)
```

51.2 Key Lookup Elimination

```
-- ORIGINAL: 8 seconds, 10,000 key lookups on CustomerID index
SELECT CustomerID, Name, Email, LastOrder FROM dbo.Customers
WHERE Country = 'US';

-- Existing index: IX_Customers_Country on (Country)
-- FIX: add INCLUDE columns to make it covering
CREATE INDEX IX_Customers_Country_Cover ON dbo.Customers (Country)
INCLUDE (Name, Email, LastOrder);

-- Result: 60 ms (130× faster); cost: slightly larger index, slower writes
```

51.3 Correlated Subquery → Window Function

```
-- ORIGINAL: 90 seconds, correlated subquery runs once per order (1M orders)
SELECT o.OrderID, o.CustomerID, o.Total,
       (SELECT AVG(Total) FROM dbo.Orders WHERE CustomerID = o.CustomerID) AS CustAvg
FROM dbo.Orders o;

-- FIX: window function (single pass)
SELECT OrderID, CustomerID, Total,
       AVG(Total) OVER (PARTITION BY CustomerID) AS CustAvg
FROM dbo.Orders;

-- Result: 2 seconds (45× faster)
```

51.4 Cursor → Set-Based UPDATE

```
-- ORIGINAL: 5 minutes, cursor updating row-by-row
DECLARE c CURSOR FOR SELECT CustomerID FROM dbo.Customers WHERE Status = 'Inactive';
OPEN c; FETCH NEXT FROM c INTO @ID;
WHILE @@FETCH_STATUS = 0
BEGIN
    UPDATE dbo.Orders SET Status = 'Archived' WHERE CustomerID = @ID;
    FETCH NEXT FROM c INTO @ID;
END
CLOSE c; DEALLOCATE c;

-- FIX: set-based UPDATE with JOIN (one statement)
UPDATE o
SET o.Status = 'Archived'
```

```
FROM dbo.Orders o
JOIN dbo.Customers c ON c.CustomerID = o.CustomerID
WHERE c.Status = 'Inactive';

-- Result: 8 seconds (40x faster)
```

51.5 Parameter Sniffing Fix

```
-- ORIGINAL: Stored proc was fast for @Status='Active' (5 rows) but 60 sec
-- for @Status='Pending' (1M rows). First-call plan cached a seek optimized
-- for low cardinality; subsequent calls reused it badly.

CREATE PROCEDURE dbo.GetOrders @Status VARCHAR(20) AS
    SELECT * FROM dbo.Orders WHERE Status = @Status;
GO

-- FIX 1: RECOMPILE on hot statement (small overhead per call)
CREATE PROCEDURE dbo.GetOrders @Status VARCHAR(20) AS
    SELECT * FROM dbo.Orders WHERE Status = @Status
    OPTION (RECOMPILE);

-- FIX 2 (better): parameter-sensitive plan optimization (SQL 2022+) or
-- two procs (one for hot path, one for cold path).

-- Result: Pending now 200ms; Active stays 5ms
```

51.6 Temp Table vs CTE for Multi-Reference

```
-- ORIGINAL: 45 seconds. The CTE was inlined and evaluated twice.
WITH Heavy AS (
    SELECT CustomerID, SUM(Total) AS Rev FROM dbo.Orders
    GROUP BY CustomerID
)
SELECT * FROM Heavy h1
JOIN Heavy h2 ON h1.Rev = h2.Rev AND h1.CustomerID < h2.CustomerID;

-- FIX: materialise once into a temp table with statistics
SELECT CustomerID, SUM(Total) AS Rev
INTO #CustRev
FROM dbo.Orders
GROUP BY CustomerID;
CREATE INDEX IX_Tmp ON #CustRev(Rev);

SELECT * FROM #CustRev h1
JOIN #CustRev h2 ON h1.Rev = h2.Rev AND h1.CustomerID < h2.CustomerID;

-- Result: 4 seconds (11x faster)
```

52. Best Practices

A consolidated checklist of MSSQL best practices across schema design, query writing, indexing, transactions, security, and operations. Following these prevents most production issues.

52.1 Schema Design

- **Use schemas, not prefixes** — Sales.Customers not dbo.usp_Sales_GetCustomers.
- **Surrogate PK + UNIQUE natural key** — INT IDENTITY for PK; UNIQUE constraint for business keys.
- **Choose the smallest data type** — TINYINT not INT for status codes; DATE not DATETIME2 for date-only; VARCHAR not NVARCHAR for ASCII-only.
- **NOT NULL by default** — Allow NULL only when genuinely optional. NULLs complicate queries and indexes.
- **Foreign keys ON** — Enforce referential integrity. Disable only for bulk loads; re-enable WITH CHECK.
- **Use constraints over triggers** — CHECK, UNIQUE, FK are simpler and faster.
- **Timestamps** — DATETIME2(0-7), not DATETIME. SYSUTCDATETIME for storage; convert at presentation.

52.2 Query Writing

- **Sargable predicates** — No functions on indexed columns in WHERE.
- **Explicit column list** — No SELECT * — list columns you need.
- **Set-based, not cursors** — Use UPDATE...FROM, MERGE, window functions.
- **Parameterise** — sp_executesql with parameters; never string-concat user input.
- **NOLOCK with care** — READ UNCOMMITTED reads dirty data. Use RCSI instead for reader/writer isolation.
- **Use EXISTS over IN for correlated** — NOT EXISTS handles NULLs correctly; NOT IN does not.
- **OFFSET/FETCH for pagination** — Or keyset for deep paging.

52.3 Indexing

- **Clustered on PK** — Sequential IDENTITY or sequential GUID.
- **Nonclustered on FK columns** — Joins/filters on FK need indexes.
- **Cover hot queries** — INCLUDE frequently-selected columns to avoid lookups.
- **Filtered indexes for sparse data** — WHERE IsActive = 1 narrows the index.
- **Drop unused indexes** — Every index slows writes. Use sys.dm_db_index_usage_stats.
- **Rebuild >30% frag, reorganise 5-30%** — Schedule with Ola Hallengren scripts.
- **Update statistics** — After large data loads; auto-update may lag.

52.4 Transactions

- **Keep them short** — Smaller blocking/deadlock window.
- **SET XACT_ABORT ON** — Always in production procs.
- **TRY/CATCH + THROW** — No swallowed errors.
- **Lowest isolation that works** — Default READ COMMITTED; consider RCSI.
- **Consistent lock order** — Prevents deadlocks.

52.5 Security

- **Windows Authentication** — Prefer over SQL logins.
- **Least privilege** — App login gets EXECUTE on procs, not db_datareader.
- **TDE for production** — Encrypt data files.
- **Always Encrypted for sensitive** — SSN, credit cards — even DBAs cannot see.
- **Audit critical tables** — SQL Server Audit feature for compliance.
- **Rotate passwords / service accounts** — Use Managed Identity in Azure.

52.6 Operations

- **Backups: 3-2-1 rule** — 3 copies, 2 media, 1 offsite. Test restores monthly.
- **Ola Hallengren maintenance** — Industry standard for index/stats/backup jobs.
- **Query Store ON** — For every production database.
- **Monitor wait stats** — sys.dm_os_wait_stats tells you where time is spent.
- **Pre-size data and log files** — Avoid autogrowth on production.
- **Instant File Initialization** — Grant Perform Volume Maintenance Tasks to SQL Service account — skips zeroing data files.
- **Fixed autogrowth sizes** — 50-100 MB data, 25 MB log. Never percentage.
- **tempdb: 1 file per CPU (up to 8)** — Same size, same autogrowth. TF 1118 on old versions.

53. Frequently Asked Queries

Practical T-SQL snippets you will be asked to write or explain in interviews. Memorise the patterns; the exact syntax you can look up.

53.1 Find Duplicate Rows

```
-- Find customers with duplicate emails
SELECT Email, COUNT(*) AS Cnt
FROM dbo.Customers
WHERE Email IS NOT NULL
GROUP BY Email
HAVING COUNT(*) > 1;

-- With row details using window function
WITH Dupes AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY Email ORDER BY CustomerID) AS Rn
    FROM dbo.Customers
)
SELECT * FROM Dupes WHERE Email IN (
    SELECT Email FROM dbo.Customers WHERE Email IS NOT NULL
    GROUP BY Email HAVING COUNT(*) > 1
) ORDER BY Email, Rn;
```

53.2 Delete Duplicates (Keep One)

```
WITH Dupes AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY Email ORDER BY CustomerID) AS Rn
    FROM dbo.Customers
)
DELETE FROM Dupes WHERE Rn > 1; -- keeps the lowest CustomerID
```

53.3 Nth Highest Salary

```
-- 3rd highest salary per department (uses DENSE_RANK for ties)
WITH Ranked AS (
    SELECT DepartmentID, Salary,
           DENSE_RANK() OVER (PARTITION BY DepartmentID ORDER BY Salary DESC) AS Rk
    FROM dbo.Employees
)
SELECT DepartmentID, Salary FROM Ranked WHERE Rk = 3;
```

53.4 Top N per Group

```
-- Top 3 highest-paid employees per department
WITH Ranked AS (
    SELECT EmployeeID, DepartmentID, Salary, Name,
           ROW_NUMBER() OVER (PARTITION BY DepartmentID ORDER BY Salary DESC) AS Rn
    FROM dbo.Employees
)
SELECT * FROM Ranked WHERE Rn <= 3 ORDER BY DepartmentID, Rn;
```

53.5 Running Total

```
-- Cumulative revenue by date
SELECT OrderDate,
       SUM(Total) OVER (ORDER BY OrderDate
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
RunningTotal
FROM dbo.Orders
ORDER BY OrderDate;
```

53.6 Year-over-Year Growth

```
WITH Yearly AS (
    SELECT YEAR(OrderDate) AS Yr, SUM(Total) AS Revenue
    FROM dbo.Orders
    GROUP BY YEAR(OrderDate)
)
SELECT Yr, Revenue,
       LAG(Revenue) OVER (ORDER BY Yr) AS PrevYearRevenue,
       Revenue - LAG(Revenue) OVER (ORDER BY Yr) AS Diff,
       CAST((Revenue - LAG(Revenue) OVER (ORDER BY Yr))
            * 100.0 / LAG(Revenue) OVER (ORDER BY Yr) AS DECIMAL(5,2)) AS PctGrowth
FROM Yearly;
```

53.7 Pivoting Rows to Columns

```
-- Total sales per month per year (pivot)
SELECT YEAR(OrderDate) AS Yr,
       [1] AS Jan, [2] AS Feb, [3] AS Mar, [4] AS Apr,
       [5] AS May, [6] AS Jun, [7] AS Jul, [8] AS Aug,
```



```

        [9] AS Sep, [10] AS Oct, [11] AS Nov, [12] AS Dec
FROM (
    SELECT YEAR(OrderDate) AS Yr, MONTH(OrderDate) AS Mo, Total
    FROM dbo.Orders
) src
PIVOT (SUM(Total) FOR Mo IN ([1],[2],[3],[4],[5],[6],[7],[8],[9],[10],[11],[12])) p;

```

53.8 Hierarchical Query (Recursive CTE)

```

WITH OrgChart AS (
    SELECT EmployeeID, Name, ManagerID, 0 AS Level,
           CAST(Name AS NVARCHAR(1000)) AS Path
    FROM dbo.Employees WHERE ManagerID IS NULL
    UNION ALL
    SELECT e.EmployeeID, e.Name, e.ManagerID, o.Level + 1,
           CAST(o.Path + ' > ' + e.Name AS NVARCHAR(1000))
    FROM dbo.Employees e
    JOIN OrgChart o ON e.ManagerID = o.EmployeeID
)
SELECT * FROM OrgChart ORDER BY Path OPTION (MAXRECURSION 100);

```

53.9 Find Missing IDs (Gaps)

```

-- Find gaps in a sequence (e.g., missing OrderIDs)
WITH Gaps AS (
    SELECT OrderID,
           LEAD(OrderID) OVER (ORDER BY OrderID) AS NextID
    FROM dbo.Orders
)
SELECT OrderID + 1 AS RangeStart, NextID - 1 AS RangeEnd
FROM Gaps
WHERE NextID - OrderID > 1;

```

53.10 Concatenate Rows into a String

```

-- Comma-separated list of products per order (2017+)
SELECT o.OrderID,
       STRING_AGG(p.Name, ', ') WITHIN GROUP (ORDER BY p.Name) AS Products
FROM dbo.Orders o
JOIN dbo.OrderItems oi ON oi.OrderID = o.OrderID
JOIN dbo.Products p ON p.ProductID = oi.ProductID
GROUP BY o.OrderID;

```

54. Advanced Topics

These topics go beyond the fundamentals. Mid-level and senior roles expect familiarity — not necessarily mastery, but awareness of when each is appropriate.

54.1 Columnstore Indexes

- **What** — Column-oriented storage with aggressive compression. 5-10× smaller than rowstore; 10-100× faster aggregates on large tables.
- **Use** — Data warehouse fact tables, analytics on wide tables, time-series aggregation.
- **Clustered columnstore** — Replaces the rowstore entirely. Table is columnar.

- **Nonclustered columnstore** — Adds columnar index on a rowstore table for hybrid OLTP+analytics.
- **Limitations** — Less efficient for point lookups; LOB columns restricted; foreign keys not supported on clustered columnstore tables.

54.2 In-Memory OLTP (Hekaton)

- **What** — Memory-optimised tables and natively compiled stored procedures. No locks, no latches, no log writes for reads.
- **Use** — High-throughput ingest (IoT, clickstream), low-latency lookups, session state.
- **Trade-offs** — Memory-bound (tables live in RAM); schema changes limited; some T-SQL features unsupported in native procs.
- **Durability** — SCHEMA_AND_DATA (default) writes log + checkpoint files; SCHEMA_ONLY rebuilds schema on restart (no data persistence).

54.3 Query Hints

Hint	Effect	Use Case
OPTION (RECOMPILE)	Recompile on each call	Volatile parameter sniffing
OPTION (OPTIMIZE FOR UNKNOWN)	Use density, not parameter value	Generic plan
OPTION (MAXDOP N)	Cap parallelism	Limit CPU for one query
OPTION (FAST N)	Optimise for first N rows	Pagination UI
OPTION (MAXRECURSION N)	Cap recursive CTE	Date series
OPTION (TABLE HINT ...)	Force a specific index or lock	Override optimizer
OPTION (HASH JOIN / MERGE JOIN)	Force join algorithm	Rare — usually optimizer knows best

Hint discipline: Hints override the optimizer — use sparingly. They are a workaround for bad plans, not a tuning technique. Each hint should have a comment explaining why it was added and when it should be re-evaluated. Statistics updates, schema redesign, and plan forcing via Query Store are usually better long-term fixes.

54.4 Service Broker

- **What** — Native message queue inside SQL Server. Reliable, transactional, asynchronous.
- **Use** — Event-driven processing inside the database, async triggers, cross-database messaging without external queues.
- **Components** — Message types, contracts, queues, services, routes.
- **Modern alternative** — External message broker (RabbitMQ, Service Bus, Kafka) is usually preferred for new systems.

54.5 Change Data Capture (CDC) and Change Tracking

Feature	What It Captures	Use Case
CDC	Tracks all changes with before/after values	ETL ingestion, audit
Change Tracking	Tracks only that a row changed (not the values)	Sync to cache, offline client

54.6 Temporal Tables (System-Versioned)

```
-- Auto-track history of every row change
CREATE TABLE dbo.Employees (
    EmployeeID INT PRIMARY KEY,
    Name NVARCHAR(100),
    Salary DECIMAL(10,2),
    SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START NOT NULL,
    SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END NOT NULL,
    PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime)
)
WITH (SYSTEM_VERSIONING = ON (HISTORY_TABLE = dbo.EmployeesHistory));

-- Query as of a point in time:
SELECT * FROM dbo.Employees
FOR SYSTEM_TIME AS OF '2026-01-01';
```

54.7 Graph Tables (SQL Server 2017+)

- **Node tables** — Entities (Person, Product, Account).
- **Edge tables** — Relationships (FriendOf, Bought, Reviewed).
- **MATCH clause** — Cypher-like query syntax: `SELECT * FROM Person p1, FriendOf f, Person p2 WHERE MATCH(p1-(f)-p2)`.
- **Use** — Social networks, recommendation engines, fraud detection.

55. HR + Project Related Questions

HR and project rounds test communication, conflict resolution, ownership, and fit. Use the STAR framework (Situation, Task, Action, Result) for every behavioural answer. Prepare 3-5 concrete stories from your SQL Server work.

55.1 STAR Framework

Letter	Cover	Example
Situation	Set context in 1-2 sentences	At Acme, our nightly ETL was missing SLA
Task	Your responsibility	I was the DBA on the data team
Action	What YOU did (not "we")	I profiled the slowest proc, found a missing index, added it, restructured the proc

Letter	Cover	Example
Result	Quantified outcome	ETL went from 6 hours to 90 minutes; SLA met

55.4 Questions to Ask the Interviewer

- What is the largest database size and the highest transaction rate the team supports?
- How is HA/DR implemented — Always On AGs, log shipping, IaaS, PaaS?
- What is the on-call rotation and how severe are typical pages?
- How does the team handle deployments — maintenance windows, blue/green, rolling?
- What is the biggest performance or scalability challenge the team is facing?
- How are migrations handled — schema changes, data backfills, version upgrades?
- What does success look like for this role in the first 6 months?
- What is the team's stance on cloud migration — Azure SQL, Managed Instance, keep on-prem?
- How is the DBA team organised — separate from developers, embedded, both?

55.5 Final Tips

- Think aloud during whiteboard queries; silent candidates look stuck.
- Ask clarifying questions before writing SQL — schema, indexes, expected output.
- Start with the simplest correct SQL, then optimise — do not jump to fancy tricks.
- For tuning scenarios, always propose measuring first (execution plan, DMVs).
- Quantify impact in behavioural answers (5 TB database, 200 ms latency, 99.99% uptime).
- Have 3-5 STAR stories ready: a hard tuning win, a production incident, a design decision, a disagreement, a leadership moment.
- Know your resume cold — anything on it is fair game.
- Sleep well the night before. A rested brain beats last-minute cramming.

Final reminder: Three years of experience means you should be able to discuss trade-offs, not just recite definitions. Whenever an interviewer asks "what is X?", answer with what it is, when to use it, and one limitation. The third sentence is what separates mid-level from junior.